Performance of Rust language

Work in progress

About us

Yuri Gribov (yugr, the_real_yugr), https://github.com/yugr

- Developer and fan of system SW (compilers, runtimes, verification tools, etc.)
- Compiler teamlead

Zakhar Akimov, https://github.com/z-inform

- Developer of general-purpose and AI compilers

Rust language features

- Language for high-performance system programming
  - Zero cost/overhead abstractions: don’t pay for what you don’t use + no (or minimal) runtime overhead (iterators, closures, traits, etc.)
  - Allows low-level tuning (SIMD, inline assembler, compiler intrinsics like __builtin_expect, etc.)
- Supports modern programming paradigms
  - Procedural, functional and object-oriented programming
  - Static and runtime polymorphism
  - Etc.
- Reduces number of UBs through stricter language rules and runtime checks
  - Especially memory safety

Rust performance: Disadvantages

- Runtime checks
- Forced initialization
- Performance-robustness tradeoffs in stdlib
  - Protect against invalid comparators, DoS-robust hashes, etc.
- Lack of -ffast-math
- Signed integer overflows are allowed
- …

Rust performance: Advantages

- More aggressive alias analysis
  - Basically restrict-by-default
- Structure layout optimizations
  - Field reorder and niche optimizations
  - Similar to -fipa-struct-reorg in GCC
- More efficient standard library containers
- ...

Goals of this talk

https://pxhere.com/ru/photo/870592

- How “expensive” are Rust checks in practice ?
  - Compared to Rust without checks and/or C/C++
  - What are the immediate (extra instructions) and indirect (disabling optimizations, I$/BTB pressure) consequences
- Can Sufficiently Smart (Heroic) Compilers © help to mitigate overheads ?
- And if not, can developer help compiler to get rid of them ?
- On the contrary, what language features help compiler to optimize code better ?

Opinion of language creators

- Steven Klabnik :
  - "Generally Rust is the same order of magnitude as C. Sometimes faster, sometimes slower”
  - “Rust prioritizes memory safety above all else. But speed is a close second"

Overhead analysis

- How can we estimate overheads caused by checks ?
- Rust compilers do not provide flags to disable safety checks
- We will compare regular Rust with patched Rust with disabled checks
  - Repo with modified compiler: yugr/rust-private
  - Baseline branch yugr/baseline (on top of 1.87.0)
    - Disabled hashtable randomization and forced function alignment for better measurement stability
- And also demonstrate overhead for similar checks in “safe” C++ dialect (Hardened C/C++)

Rust and C++

- Rust and C++ are similar in expressive power:
  - Support several programming paradigms (procedural, OOP, functional)
  - Different types of polymorphism
  - Etc.
- Similar application domain (high-performance system programming)
  - Zero cost/overhead abstractions
  - Support for low-level tuning
- Same backend optimizers (LLVM, GCC) and optimization technology (PGO, LTO, etc.)

Not covered: Other language aspects

- Ergonomics, expressiveness (e.g. self-referential data structures), safety, etc.
- A lot of talks on these topics already !
  - Rust Features that I Want in C++ (CppNow 2022)
  - Rust vs C++ with Steve Klabnik and Herb Sutter (SW Engineering Daily podcast)
  - The existential threat against C++ and where to go from here (NDC 2024)

Not covered: Parallel programming

- Easy and reliable parallel programming is considered one of the big advantages of Rust
  - Fearless Concurrency with Rust
- But this requires a separate talk and deeper expertise
  - RustCon 2027 anyone ?

Not covered: Non-idiomatic code

- We only consider “typical” Rust code
  - No SIMD or compiler intrinsics, inline assembler, unsafe (including get_unchecked or wrapping_add), restrict, etc.
  - Standard library
- With unsafe, etc. (and lots of free time) one can always achieve identical Rust and C++ performance

Not covered: Non-idiomatic code

- Nikita Popov, LLVM lead maintainer:
  - “Languages like C, C++ and Rust are by-design on equal footing“
  - “You can only compare how easy it is to write performant code or how performant idiomatic code is”
- Scott McMurray, Rust compiler team:
  - “Clang and rustc are both producing LLVM IR and running mostly the same LLVM optimization passes”
  - “Given enough work they can always produce essentially the same thing, making the comparisons pointless”

Bounds checks

Detailed analysis

Buffer overflows

- Morris Worm (1988)
- Writing excessively large volume of data into program variable

```
char local_buf[32];
sprintf(buf, "Message from user: %s", received_data);
```

- Stack overflow
  - Stack Smashing, Return-to-libc, Return-Oriented Programming attacks
  - The most standardized and widely used type of attacks
- Heap overflow

https://www.rawpixel.com/image/5958324/free-public-domain-cc0-photo

Prevalence of buffer overflow vulnerabilities

- Leads ratings of most dangerous vulnerabilities
  - Mitre CWE Top 25 2024 (2, 6, 20 places)
- 70% vulnerabilities in Microsoft products and Chromium caused by buffer overflows
- 40% of memory-error-based attacks exploit buffer overflows
  - Google Project Zero(2024)
- 80% of Memory Safety CVEs and 46% of Memory Safety KEVs in 2024
  - 20%+ are stack overflows (most dangerous vulnerability)
  - May not be the best metric (most CVEs are web vulnerabilities)

Memory safety in Rust

- Frontend generates bounds checks for standard index accesses
  - Arrays, slices, containers, etc.
  - Checks may be eliminated later if optimizer can guarantee correctness
- Uses safer algorithms in stdlib
  - E.g. sorting won’t crash on incorrect comparators

https://www.picpedia.org/highway-signs/s/safety.html

Overheads

- Comparison and non-taken conditional jump (2 or more cycles)
- Extra register for keeping object length for comparison
- I$ pressure (cache misses) due to checking code and panic handlers
- BTB pressure (branch mispredictions)
  - Adding a (predictable) branch to existing code can increase branch mispredictions (Daniel Lemire)
    - 2-4x increase of mispredictions due to *predictable* branch
  - LeetCode fans know that resize + operator[] is significantly faster than push_back (see push-dominoes)
- Disabling other optimizations
  - Mainly vectorizer

Compiler optimizations

- In many cases compiler can
  - Prove correctness of memory access and remove checks
  - Or extract checks from loops (O(N) -> O(1))
- Such optimizations are done in LLVM optimizer
  - InstCombine, SimpleLoopUnswitch, LoopVectorize, etc.
  - Also help Hardened C/C++
- Optimizer has been significantly improved and many older issues are successfully optimized in modern Rust
  - E.g. well-known examples of Alex Kladov (matklad)

Successful optimization

All checks moved to function prologue

```rust
#[no_mangle]
pub fn foo(a: &[i32], b: &[i32]) -> i32 {
    let mut ans = 0;
    let n = a.len();
    for i in 0..n {
        ans += a[i] * b[i];
    }
    ans
}
```

```
foo:
        .cfi_startproc
        test    rsi, rsi
        je      .LBB0_1   // n == 0 => return 0
        lea     rax, [rsi - 1]
        cmp     rcx, rax
        jbe     .LBB0_6   // b.len() < n => panic
        xor     eax, eax
        xor     ecx, ecx
        .p2align        4
.LBB0_4:                  // Loop (without checks!)
        mov     r8d, dword ptr [rdx + 4*rcx]
        imul    r8d, dword ptr [rdi + 4*rcx]
        lea     r9, [rcx + 1]
        add     eax, r8d
        mov     rcx, r9
        cmp     rsi, r9
        jne     .LBB0_4
...
```

Compiled with -Cno-vectorize-loops -Cno-vectorize-slp -Cllvm-args=-unroll-allow-remainder=0

Unsuccessful optimization

Checks remain within the loop

```rust
#[no_mangle]
pub fn foo(v: &[i32]) -> i32 {
    let mut ans = 0;
    let chunk_len = 4;
    let num_chunks = v.len() / chunk_len;
    for i in 0..num_chunks {
        for j in 0..chunk_len {
            ans += v[i * chunk_len + j];
        }
    }
    ans
}
```

Compiled with -Cno-vectorize-loops -Cno-vectorize-slp -Cllvm-args=-unroll-allow-remainder=0

```
bar:
        ...
        .p2align        4
.LBB0_2:        // Loop (with checks!)
        lea     rcx, [r8 - 3]
        cmp     rcx, rsi
        jae     .LBB0_7
        lea     rcx, [r8 - 2]
        cmp     rcx, rsi
        jae     .LBB0_7
        lea     rcx, [r8 - 1]
        cmp     rcx, rsi
        jae     .LBB0_7
        cmp     r8, rsi
        jae     .LBB0_6
        add     eax, dword ptr [rdi + 4*r8 - 12]
        add     eax, dword ptr [rdi + 4*r8 - 8]
        add     eax, dword ptr [rdi + 4*r8 - 4]
        add     eax, dword ptr [rdi + 4*r8]
        add     r8, 4
        dec     rdx
        jne     .LBB0_2
```

Problem with optimizer: delayed panics

- Rust RFC 560 suggested that compiler could combine several checks together
  - If it helps optimization
  - And observed program behavior won't change (modulo error message)
- This has not been decided and optimization currently isn't enabled for bounds checks ☹
- Example: rust #50759

```
use std::fs::{read, write};

fn main() -> Result<(), std::io::Error> {
    let h = read("/tmp/test")?;
    // Each access checked separately ☹
    // (would be enough to check only h[15])
    let b = [h[0], h[1], ... h[15]];
    write("/tmp/test", &b)?;
    Ok(())
}
```

Workarounds: Overview

- Use slices (rather than containers) in loops
  - This simplifies job of alias analysis
  - Example: fschutt/fastblur #3
- Avoid complex index computations
  - Anything more complicated than adding a constant or adding two variables
  - Even affine operations may break optimizer: Taking Advantage of Auto-Vectorization in Rust
- Do not rely on optimizer in non-trivial cases
  - Optimizations vary (non-monotonically) across compiler versions
  - "Auto-vectorization is not a programming model"
- Same advices are relevant for Hardened C/C++ !

Workarounds

- Checks elimination
- Reducing number of checks
- Reducing cost of checks

Workarounds

- **Checks elimination**
- Reducing number of checks
- Reducing cost of checks

Workarounds: Unsafe-code

- The simplest and most robust solution
- Used in stdlib containers and iterators
- Several variants:
  - Explicit pointers in unsafe blocks
  - APIs like get_unchecked to disable check
  - Compiler hints core::hint::unreachable_unchecked and core::hint::assert_unchecked
- Example: [rust #36072]

```
    /// Return a reference to the element at `index`.
    ///
-   /// Panics if the index is out of bounds.
-   ///
-   /// Unsafe because index must not equal pos.
-   #[inline(always)]
+   /// Unsafe because index must be within the data slice and not equal to pos.
+   #[inline]
    unsafe fn get(&self, index: usize) -> &T {
        debug_assert!(index != self.pos);
-       &self.data[index]
+       debug_assert!(index < self.data.len());
+       self.data.get_unchecked(index)
    }
```

Workarounds: Bounded types

- Index range can be expressed via type system:
  - Enums or bool
- Example: [rust #102303]

```
fn foo(array: &[i16; 3],
       i: usize) -> i16 {
    array[i]
}
```

```
pub enum Enum {
    A, B, C
}

fn foo(array: &[i16; 3],
       i: Enum) -> i16 {
    array[i as usize]
}
```

Workarounds

- Checks elimination
- **Reducing number of checks**
- Reducing cost of checks

Workarounds: Manual asserts

- Replaces many checks with one
- Example: [jpeg-decoder #167]

```
+     // optimizer hint to eliminate bounds checks within loops
+     assert!(coefficients.len() == 64);
+
      let mut temp = [Wrapping(0); 64];

      // columns
      for i in 0..8 {
          if coefficients[i + 8] == 0
              && coefficients[i + 16] == 0
              && coefficients[i + 24] == 0
              && coefficients[i + 32] == 0
              && coefficients[i + 40] == 0
              && coefficients[i + 48] == 0
              && coefficients[i + 56] == 0
          {
```

Workarounds: Iterators

- Many iterators know container size and can utilize this information to remove bounds checks

```rust
pub fn bro(x: &mut [i32], y: &[i32]) {
    let n = x.len();
    for i in 0..n {
        x[i] ^= y[i];
    }
}
```

```
.LBB0_2:
        cmp     rcx, rax
        je      .LBB0_5
        mov     r8d, dword ptr [rdx + 4*rax]
        xor     dword ptr [rdi + 4*rax], r8d
        lea     r8, [rax + 1]
        mov     rax, r8
        cmp     rsi, r8
        jne     .LBB0_2
```

```rust
pub fn bro(x: &mut [i32], y: &[i32]) {
    x.iter_mut()
        .zip(y.iter())
        .for_each(|(x, y)| *x ^= *y);
}
```

```
.LBB0_2:
        mov     ecx, dword ptr [rdx + 4*rax]
        add     dword ptr [rdi + 4*rax], ecx
        lea     rcx, [rax + 1]
        mov     rax, rcx
        cmp     rsi, rcx
        jne     .LBB0_2
```

Compiled with
-Cno-vectorize-loops

Disadvantages of iterators

- “Miracle of zero cost abstraction has a limit" (fridsun at Reddit)
- Significant slowdown and code size increase for debug builds
- Compiler sometimes fails to inline long iterator chains
- Iterator lambdas take references to container elements which may cause issues with alias analysis (rust #106539)
- Chained iterators (chain, flat_map, flatten, etc.) may generate inefficient code in for-loops (“external iteration”)
  - Use only with .for_each (“internal iteration”)
  - Are iterators even efficient?

Workarounds: Reslicing

- Reslicing, preslicing or subslicing
- Pre-construct slices of known size
- Forces compiler to perform *single* check when creating slice
- Example: [dropbox/rust-brotli]

```
for i in 0..vec.len() {
    black_box(vec[i]);
}
```

→

```
let slice = &vec[..vec.len()];
for i in 0..vec.len() {
    black_box(slice[i]);
}
```

Workarounds: Slicing

- Offsetted slices should better be constructed in two steps to avoid potential integer overflow
- Example: [Optimizing rav1d, an AV1 Decoder in Rust]

```
&vec[i..(i + n)]  ──────────►  &vec[i..][..n]
```

Workarounds: Order of accesses

- Prefer to access data starting with farthest element

```
let a = x[0];
let b = x[1];
let c = x[2];
```

→

```
let c = x[2];
let b = x[1];
let a = x[0];
```

or even

```
let a = x[0];
let b = x[1];
let c = x[2];
```

→

```
let [a, b, c] = data[..3] else {
    panic!()
};
```

Workarounds

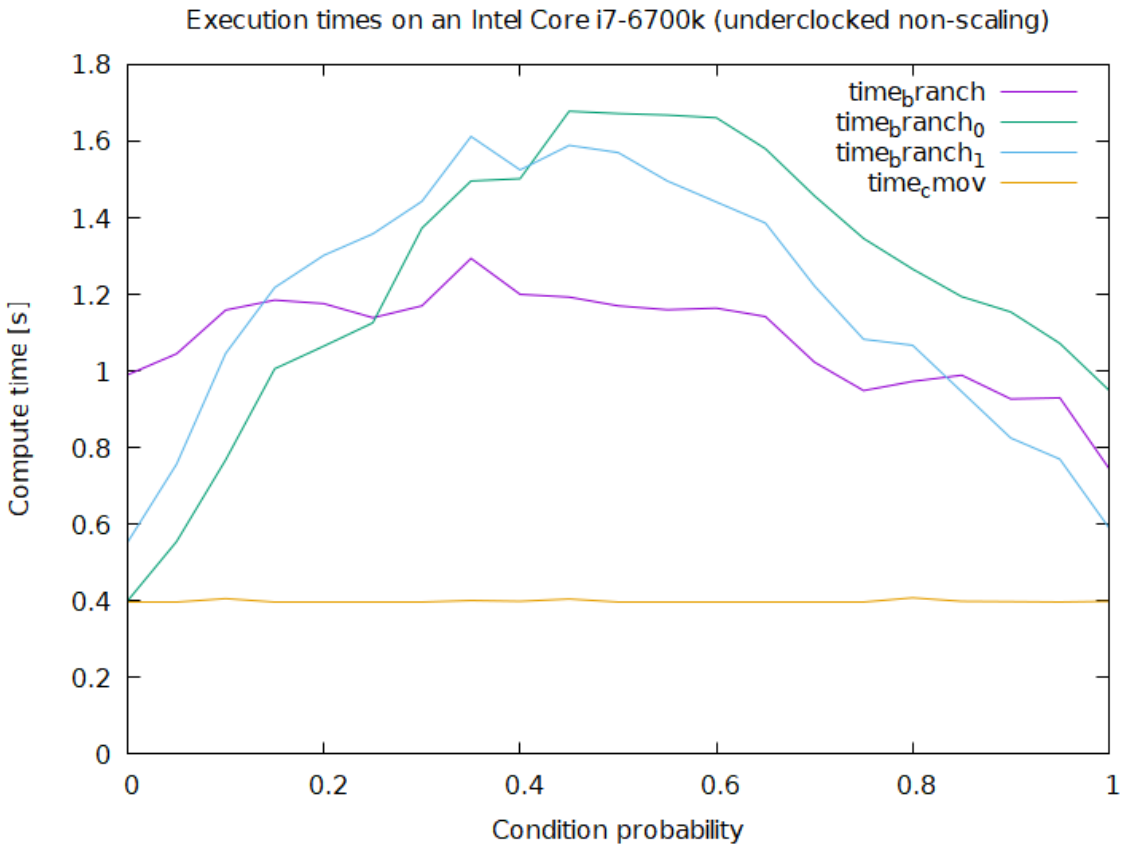
- Checks elimination
- Reducing number of checks
- **Reducing cost of checks**

Workarounds: Arithmetic instead of checks

- Checks may be replaced with conditional move

```
let i = cmp::min(i, sums.len());
sums[i] = ...
```

- CMOV is *usually* faster than branch
- Disadvantage: bounds errors will now be ignored



https://github.com/marcin-osowski/cmov

Workarounds: Arithmetic instead of checks

- Bounds checks may be replaced with masking if length is 2^N (or can be padded to 2^N):

```
const SIZE: usize = 100;
const LOOKUP_TABLE_SIZE: usize = SIZE.next_power_of_two();

let table: [u64; LOOKUP_TABLE_SIZE] = ...
... table[table & (LOOKUP_TABLE_SIZE - 1)] ...
```

- AND is *usually* faster (e.g. can run on more ALUs in processor)
- Unfortunately optimization (currently) applicable only to arrays

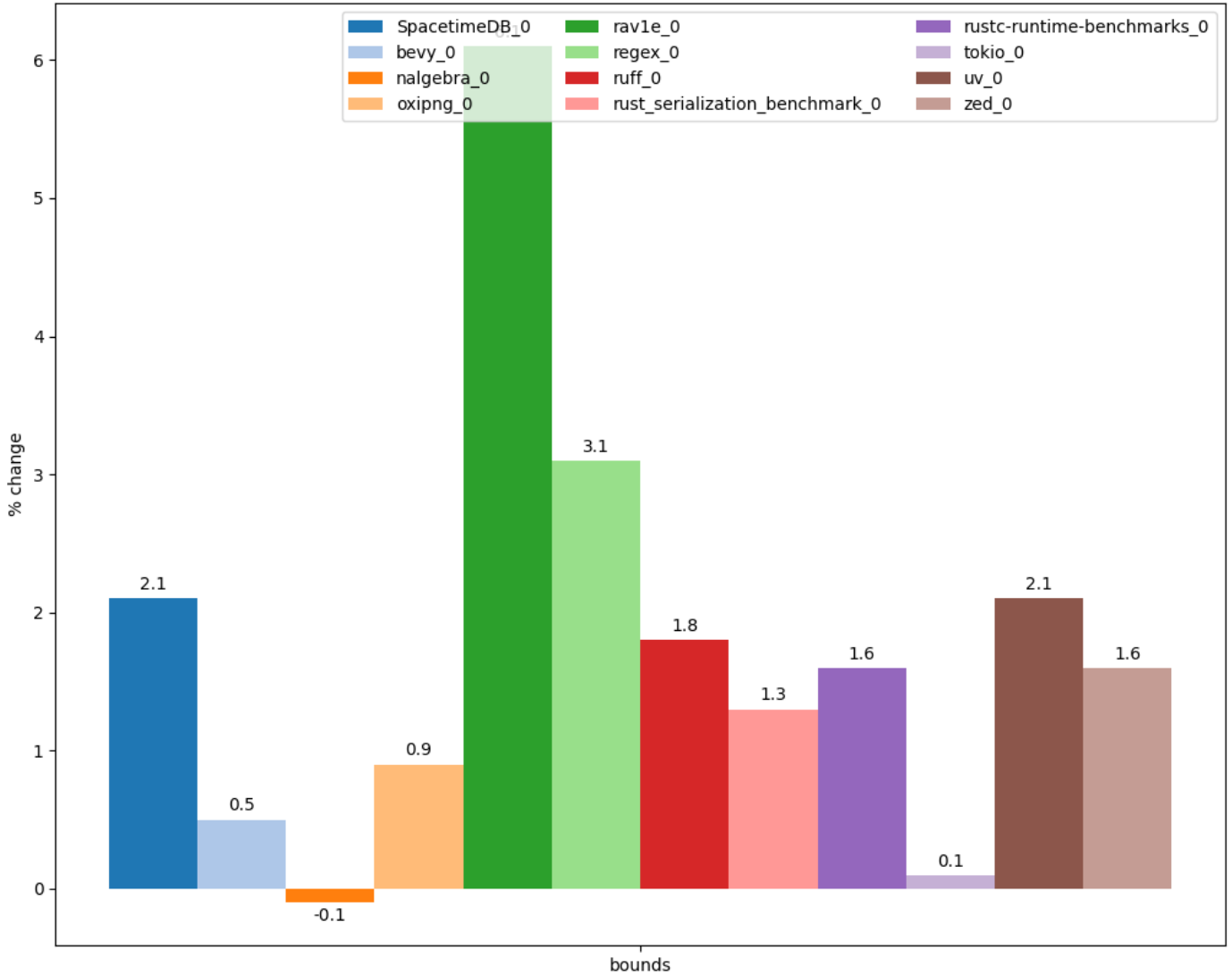
Prevalence

- More than **20%** calls to panic handlers in librustc_driver.so (Rust compiler core) caused by bounds checks
  - Based on analysis of binary code
- Disabling bounds checks reduces number of innermost loops with panic handlers **5x**
  - Data collected with LLVM plugin based on rustc and oxipng analysis

Analysis methodology

Compiler branch

Performance improvement after disabling bounds checks



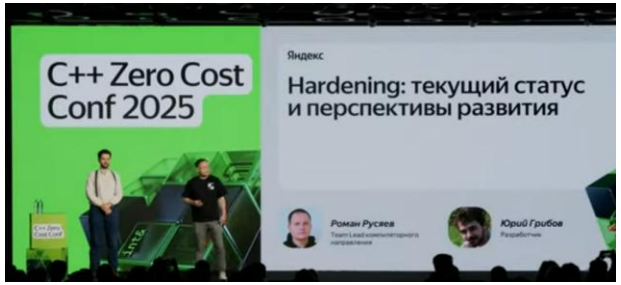
We disabled instrumentation in user code and checks in stdlib (containers, slices, strings, etc.)

Compiler branch

Hardened C/C++

- C/C++ provides limited support for memory safety:
  - StackProtector
  - _FORTIFY_SOURCE
  - Hardened STL (C++ only)
  - -fsanitize=bounds, -fsanitize=object-size
- Complete, Rust-like correctness checks are impossible due to lack of information about lengths for raw pointers
  - E.g. only 20% of non-trivial memory accesses in Clang code can be checked
  - C++ Safe Buffers may solve this but requires non-trivial code mofications
- More information:
  - Hardening: current status and trends

Mario Simoes, https://snl.no/Mount_Everest



Rust vs. Hardened C++

```rust
#[no_mangle]
pub fn foo(x: &[i32], i: usize) -> i32 {
    x[i]
}
```

rustc -O -Cpanic=abort

```
foo:
        cmp     rdx, rsi
        jae     .LBB0_2
        mov     eax, dword ptr [rdi + 4*rdx]
        ret
.LBB0_2:
        push    rax
        lea     rax, [rip +
.Lanon.acbd18db4cc2f85cedef654fccc4a4d8.1]
        ...
call    qword ptr [rip +
_ZN4core9panicking18panic_bounds_check@GOTPCREL]
```

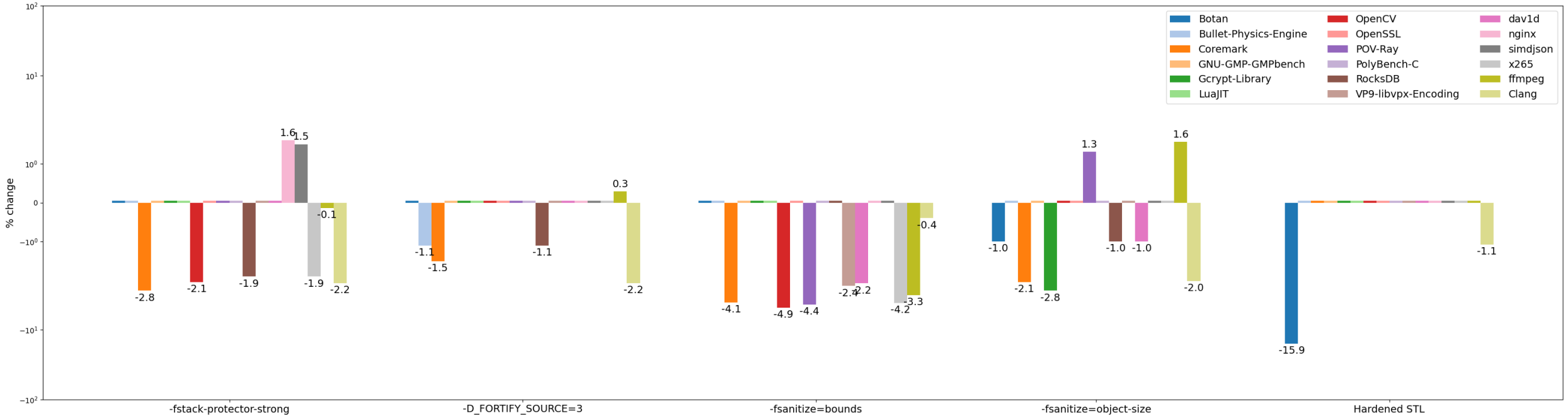
```chinese_word_
#include <span>

extern "C"
int foo(std::span<int> s, size_t i) {
  return s[i];
}
```

gcc -O2 -D_GLIBCXX_ASSERTIONS

```
foo:
.LFB821:
        cmp     rdx, rsi
        jnb     .L7
        mov     eax, DWORD PTR [rdi+rdx*4]
        ret
.L7:
        sub     rsp, 8
        ...
        call
_ZSt21__glibcxx_assert_failPKciS0_S0_@PLT
```

Overhead in C++



Unexpected performance improvements due to optimizer anomalies (bad vectorization, enabled inlining, etc.)

Conclusions

- Bounds checks in Rust are applied much more aggressively than in Hardened C/C++
  - Hardened C/C++ checks 5x accesses than Rust
- Overheads may reach 3-4% (and are comparable to Hardened C/C++)
  - But some tests may degrade by 10% and more (same applies to Hardened C/C++)
- There are many ways to avoid checks in hot code
  - Some of them won't be needed if delayed panics are finally implemented (rust #50759)

Recommended readings

- How to avoid bounds checks in Rust without unsafe (Sergey Davidoff aka Shnatsel, Rust Secure WG lead)
  - “Bible” of bounds check elimination
- Story-time: C++, bounds checking, performance, and compilers (Chandler Carruth, SW foundations & PLs technical lead, creator of Carbon language)

Aliasing rules

Detailed analysis

Pointer aliasing

- Universal feature in programming languages
- Can two different references (pointers) address same object in program ?
- Different languages make different choices:
  - No two pointers in program may refer to same data (e.g. Fortran 77)
  - Any two pointers may address same data (e.g. C89)
  - Intermediate variants
    - E.g. strict (type-based) aliasing and restrict annotations in C99

Why aliasing is important ?

- Unlimited aliasing is convenient for developer but reduces optimization opportunities for compiler
- Reduced aliasing enables many optimizations
  - E.g. vectorizers, CSE, GVN, LICM, DSE, etc.
- Alias Analysis is one of the main components of any mature compiler
  - Identifies pointers that may (or may not) alias

```c
int alias(int *a, int *b) {
  *a = 0;
  *b = 1;
  return *a;
}

int noalias(int * restrict a, int * restrict b) {
  *a = 0;
  *b = 1;
  return *a;
}
```

```
alias:
  mov     DWORD PTR [rdi], 0
  mov     DWORD PTR [rsi], 1
  mov     eax, DWORD PTR [rdi]
  ret

noalias:
  mov     DWORD PTR [rdi], 0
  xor     eax, eax
  mov     DWORD PTR [rsi], 1
  ret
```

Alias analysis theory

- Precise interprocedural alias analysis is undecidable and precise intra-procedural analysis is NP-complete
  - Undecidability of static analysis (Landi, 1992)
- Aliasing is very rare in real programs
  - 98% of pointers in SPEC benchmarks address only one object
    - Dynamic Points-To Sets (Mock et al., 2001)
- Existing alias analyses are conservative and overestimate aliasing 1.5-20x
  - Dynamic Points-To Sets (Mock et al., 2001)
  - Speculative Alias Analysis for Executable Code (Fernandez et al., 2002)
  - ORAQL — Optimistic Responses to Alias Queries in LLVM (Doerfert et al., 2023)

Aliasing in Rust

- Borrow checker rules influence performance:
  - Mutable reference may not alias any other (mutable or shared) reference
  - While it exists, mutable reference is the only pointer to object
  - Object addressed by shared reference may not be changed (while reference exists)
- In practice this achieves similar effect as restrict in C/C++

```
pub fn foo(a: &mut i32,
           b: &mut i32) -> i32 {
    *a = 0;
    *b = 1;
    *a
}
```

```
define i32 @foo(
  ptr noalias ... %a,
  ptr noalias ... %b) {
    store i32 0, ptr %a
    store i32 1, ptr %b
    ret i32 0
}
```

```
foo:
    mov     dword ptr [rdi], 0
    mov     dword ptr [rsi], 1
    xor     eax, eax
    ret
```

Limitations

- Information about aliasing is represented via noalias attribute in function arguments
  - References that appear *inside* functions are not marked and can alias from LLVM standpoint
- May be solved via alias.scope metadata in future (#16515)
- Workaround: create dummy-functions specifically for noalias annotations
  - Example: rustc

```rust
pub fn foo(a: *mut i32,
           b: *mut i32) -> i32 {
    let a = unsafe { &mut *a };
    let b = unsafe { &mut *b };
    *a = 1;
    *b = 2;
    *a
}
```

```asm
foo:
    mov     dword ptr [rdi], 1
    mov     dword ptr [rsi], 2
    mov     eax, dword ptr [rdi]
    ret
```

Limitations

- One case of above problem is aliasing of references to structure fields
- E.g. when passing two vectors to function compiler fails to understand that their buffers do not alias
- Workaround: pass raw slices to functions

```
pub fn foo(a: &mut Vec<i32>,
           b: &Vec<i32>) {
    let n = a.len();
    let b = &b[..n];
    for i in 0..n {
        a[i] = b[i] + 100;
    }
}
```

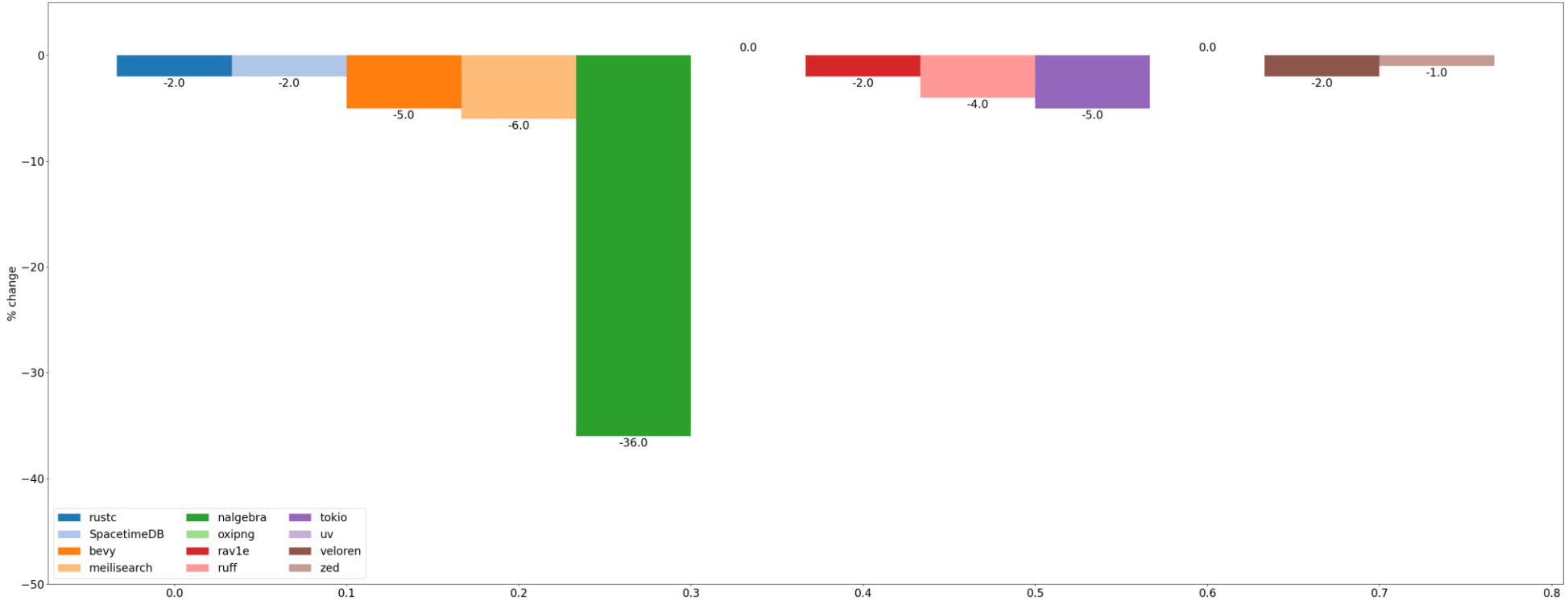
↓

```
pub fn foo(a: &mut [i32],
           b: &[i32]) {
    let n = a.len();
    let b = &b[..n];
    for i in 0..n {
        a[i] = b[i] + 100;
    }
}
```

Limitations

- Aliasing rules apply only to references
- Raw pointers will still alias in Rust
  - This is worse than in C/C++ due to lack of strict aliasing rules

Efficiency of Rust aliasing rules



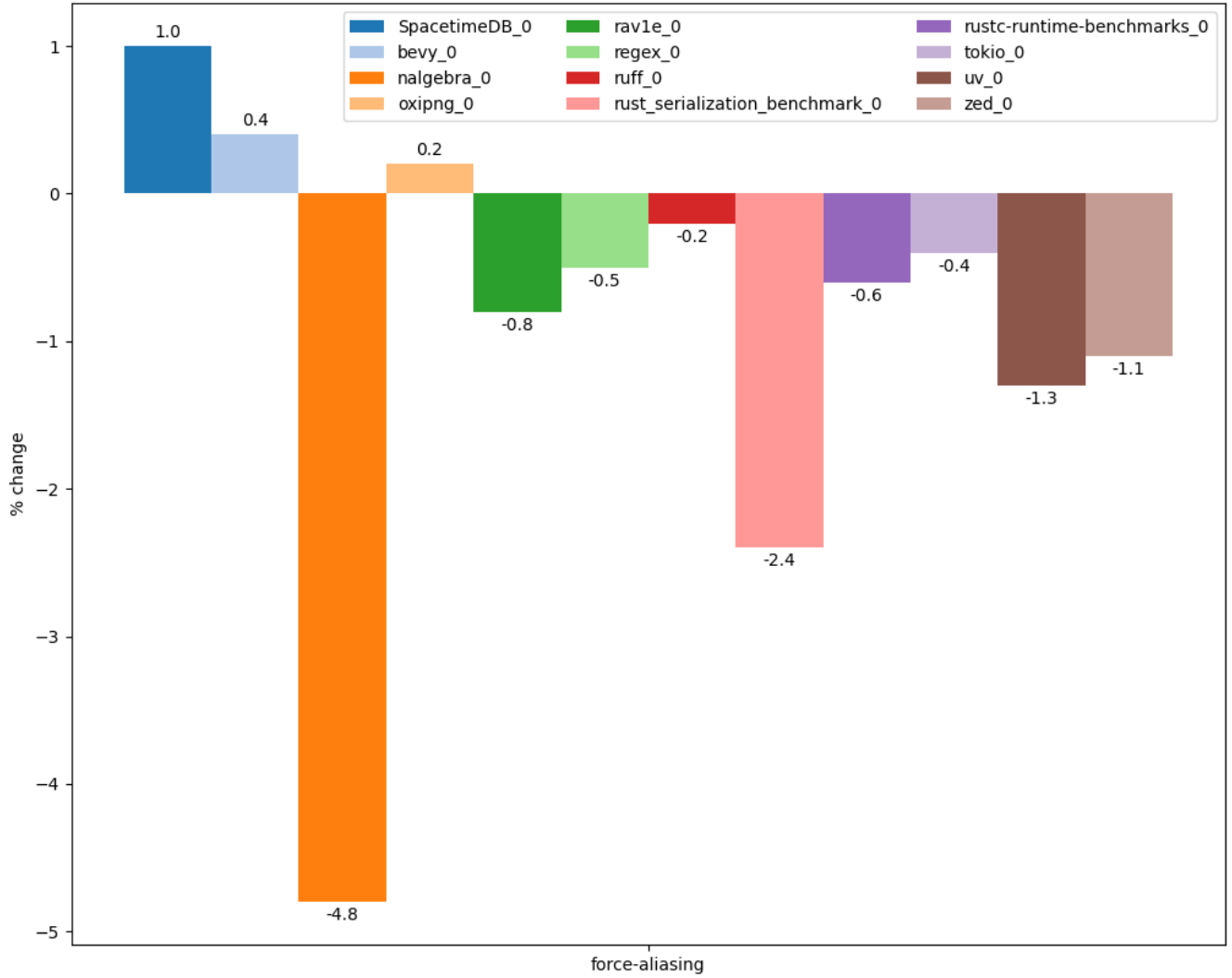
Precision of alias analysis – number of pointer pairs for which analysis could give definite answer (MustAlias / NoAlias)

Degradation of alias analysis precision in Rust after turning of reference rules

Analysis methodology

Compiler branch

Performance degradation after disabling aliasing rules



Compiler branch

Situation in C/C++

- C and C++ disallow aliasing of pointers to different types
  - Except char *
- But many projects ignore this rule (via -fno-strict-aliasing flag)
- C and C++ also allow developer to explicitly prohibit aliasing via restrict keyword
  - In practice this is seldomly used (Rust couldn’t enable its aliasing rules for long time due to lots of hidden bugs in LLVM)

Conclusions

- Rust aliasing rules allow to
  - Increase analysis precision by 5% *on average*
  - Improve performance by 1-4%
- Can expect significant performance increase after issue with aliasing metadata in rust #16515 is resolved

Recommended readings

- Complexity of alias analysis:
  - Undecidability of static analysis (Landi, 1992)
- Aliasing in real code:
  - Dynamic Points-To Sets (Mock et al., 2001)
  - Speculative Alias Analysis for Executable Code (Fernandez et al., 2002)
  - ORAQL — Optimistic Responses to Alias Queries in LLVM (Doerfert et al., 2023)

Mandatory initialization

Detailed analysis

Mandatory initialization

- Uninitialized variables may cause information leaks and other errors
  - Password or address leaks (the latter compromises ASLR)
- Prevalence:
  - 10% CVE root cause in Microsoft products in 2018 (see Killing Uninitialized Memory)
  - 12% exploitable bugs in Android (see P2723)

Example

```
void foo() {
  char password[32];
  ...
}
void bar() {
  char message[1024];
  if (cond) strcpy(message, "...");
  // Leaking password under !cond
  printf(message);
}
void baz() {
  foo();
  bar();
}
```

Rust approach

- Rust requires developer to initialize all variables
- Each variable must be assigned on all paths before first use

```
pub fn foo() {
  let x = 10;
  x
}
```

OK

```
pub fn foo(cond: bool) -> i32 {
  let mut x;
  if cond {
    x = 10;
  } else {
    x = 20;
  }
  x
}
```

OK

```
pub fn foo(cond: u32) -> i32 {
  let mut x;
  if cond > 10 {
    x = 10;
  } else if cond <= 10 {
    x = 20;
  }
  x
}
```

NOT OK

Other languages

- Many other languages also require initialization (or autoinitialize with zeroes):
  - C#, Swift, Go, Java
  - Hardened C/C++ (-ftrivial-auto-var-init), C++26 (P2795)
    - Enabled by default in Android user/kernel-space and Chrome
    - Only local varibles (global variables and sbrk/mmap-buffers already zero-initialized, heap can be zero-initialized via hardened allocator)

Issue with mandatory initializations

- Initializations are often redundant in practice
  - Will be overwritten with other values later without being read
- Compiler can’t always eliminate such redundant initializations
- Example:

```
let mut file = File::open("example.bin")?;

while (...) {
    let mut buffer = [0u8; 1024];  // memset not removed

    let n = file.read(&mut buffer)?;

    ...
}
```

Prevalence of redundant initializations

- How many initializations are redundant ?
- Static estimate (based on LLVM plugin):
  - Hardened Clang: +4% stores, 11x (!) memset calls in loops in Clang
- Dynamic estimate for rustc and clang (based on Valgrind plugin):
  - Compiling oxipng with rustc:
    - 9% unused writes
  - Compiling LICM.ii with Clang:
    - 11% unused writes
  - Compiling LICM.ii with Hardened Clang (-ftrivial-auto-var-init):
    - 23% unused writes (i.e. 2x increase)

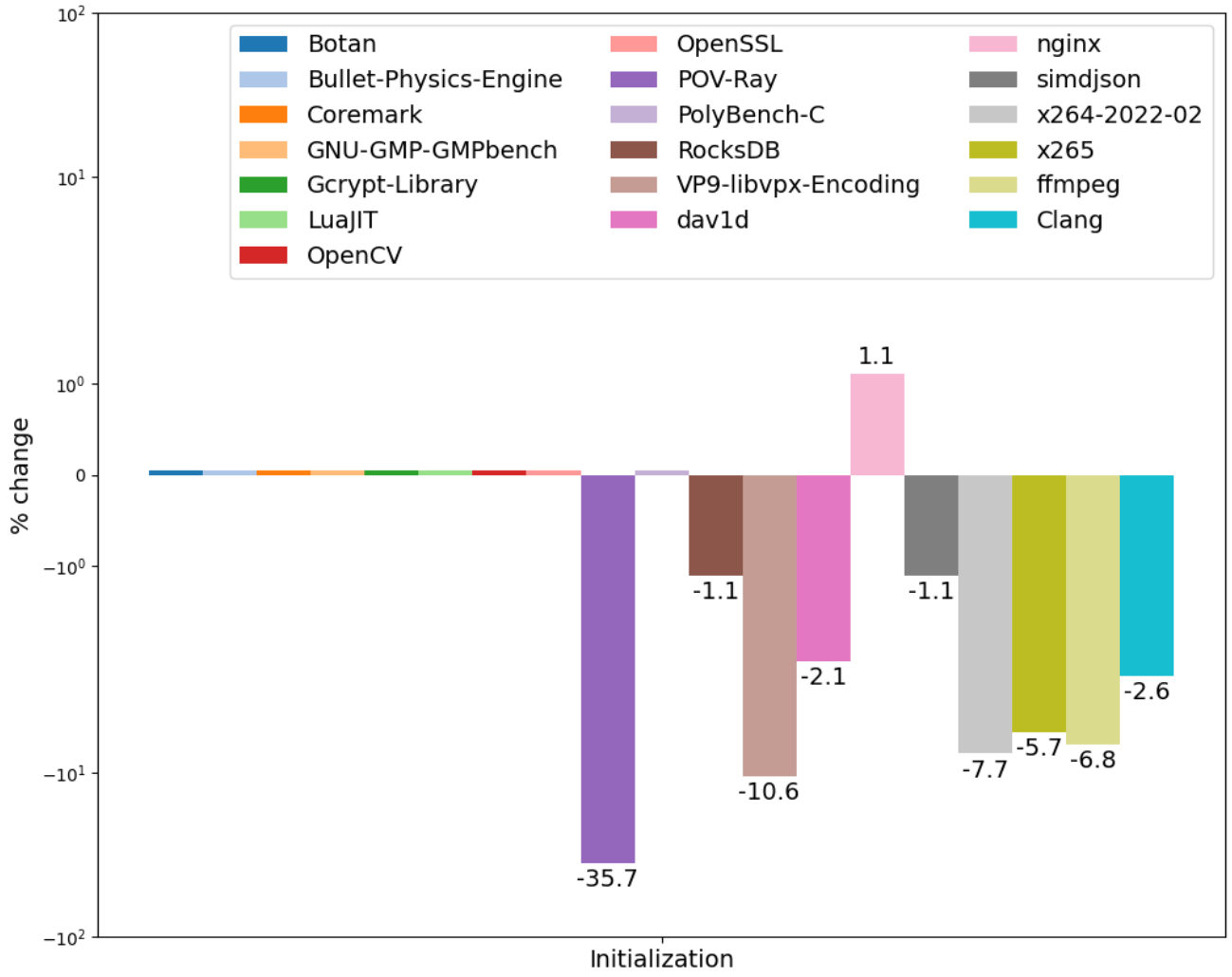
Overheads in benchmarks

- Unfortunately it’s not possible to directly measure initialization overheads in Rust benchmarks
  - That would require extensive code rewrites
- We will illustrate overheads
  - Reported in various Rust projects
  - Measured for Hardened C/C++

Example overheads in Rust programs

- What are the potential consequences for performance ?
  - 7% in stdlib file IO (rust #26950)
  - 20-30% in Hyper HTTP (see tokio #1744)
  - 30% in Shadow simulator (shadow #1643)
  - 25% in stdlib TcpStream (RFC #837)
  - 1.5% in rav1d (Making the rav1d Video Decoder 1% Faster)

Overheads with forced initialization for C++



Compiler optimizations

- Redundant scalar initializations can be optimized away by many SSA passes in LLVM (and are very cheap anyway)
  - DCE, BDCE, ADCE, InstCombine, etc.
- Redundant array initializations pose the main problem
  - DSE (DeadStoreElimination), MoveAutoInit, MemCpyOptimizer
  - To a lesser degree InstCombine

Workarounds

- Unfortunately APIs for uninitialized memory (MaybeUninit) are not very convenient
- It's not possible to simply create a reference to uninitialized data and fill it:

  "Creating a reference with &/&mut is **only allowed if** the pointer is properly aligned and **points to initialized data**" (addr_of_mut)

Workarounds: Standard types

Arrays:

```
let buf :: [u8; 4] = unsafe {
    let mut buf = MaybeUninit::<[u8;
4]>::uninit();
    let ptr = buf.as_mut_ptr();
    for i in ... {
        ptr.offset(i).write(...);
    }
    buf.assume_init()
};
```

Structures:

```
let role :: Role = unsafe {
    let mut uninit =
MaybeUninit::<Role>::uninit();
    let ptr = uninit.as_mut_ptr();
    // Need write
    // (instead of explicit assign)
    // to avoid dropping
    // uninitialized String
    addr_of_mut!((*ptr).name)
        .write("basic".to_string());
    (*ptr).flag = 1;
    (*ptr).disabled = false;
    uninit.assume_init()
};
```

Workarounds: Vectors

```
let mut buf = Vec<u8>::new();

let old_len = buf.len();
buf.reserve(data.len());

unsafe {
    let ptr = buf.spare_capacity_mut();
    for i in ... {
        ptr[idx].write(...);
    }
    buf.set_len(len + data.len());
}
```

Workarounds: IO

```
let mut buf = [MaybeUninit::uninit(); 4096];
let mut buf = BorrowedBuf::from(&mut buf[..]);
rd.read_buf(buf.unfilled())?;
if buf.len() == 0 {
    break Ok(());
}
// Work with buf.filled()
```

Conclusions

- Initialization overheads in Rust, Hardened C++ and C++26 are comparable
  - But potentially higher in Rust because it initialized heap allocations as well
- These overheads can be large in some scenarios (>10%) and require manual code tuning:
  - Using dedicated APIs like MaybeUninit

Recommended readings

- Killing Uninitialized Memory (video)
  - Report from Microsoft about enabling similar checks in Visual Studio
- P2795: Erroneous behavior for uninitialized reads
  - To be integrated to C++26

Symbol localization

Detailed analysis

Symbol localization

- Functions in translation unit (crate in Rust, cpp file in C++) can be used locally or globally (in other translation units)
  - Controlled with static keyword and anonymous namespaces in C++
  - Or with pub(crate) and pub(super) in Rust
- Rust and C++ make opposing default choice:
  - In Rust functions are local for their module by default (top-level functions are local to their crate)
  - pub(crate)-functions are local to crate
  - Only pub-functions are visible across whole program

Localization in Rust

```rust
#[inline(never)]  // Simulate inliner fail
pub(crate) fn foo1(x: i32) {
    println!("Hello world {}", x);
}

mod mymod {
    #[inline(never)]  // Simulate inliner fail
    pub(super) fn foo2(x: i32) {
        println!("Goodbye world {}", x);
    }

    #[inline(never)]  // Simulate inliner fail
    pub fn foo3(x: i32) {
        println!("Nice world {}", x);
    }
}

pub use mymod::foo3;

pub fn bar() {
    foo1(1);
    mymod::foo2(2);
    mymod::foo3(3);
}
```

```
$ rustc +baseline test.rs -O --emit=asm
--crate-type=rlib -o- \
    | rustfilt \
    | grep globl
        .globl  test::mymod::foo3
        .globl  test::bar
```

Only foo3 and bar are exported !

Compiler optimizations

- Local functions are much better optimized by compiler
- Usually such optimizations are related to API or ABI changes:
  - Interprocedural constant propagation (IPSCCP)
  - Replacing pass-by-reference with pass-by-value (ArgumentPromotion)
  - Removing dead arguments or return values (DeadArgumentElimination)
  - Relaxed calling convention (using coldcc in Transforms/IPO/GlobalOpt.cpp)
  - Interprocedural register allocation (IPRA)
- And also
  - More aggressive inlining (e.g. local functions which are only called once are always inlined)
  - Better dead code elimination

Linker optimizations

- For shared libraries (DLLs) local functions
  - May be called directly (bypassing PLT/GOT)
  - Do not need to be resolved by loader at startup
- See more details in Dynamic libraries and how to optimize them

Example

```rust
#[inline(never)]  // Simulate inliner fail
fn foo(b: bool, seed: i32, s: &[i32]) -> i32 {
    let mut ans = seed;
    for &x in s {
        ans += if b { 1 } else { ans ^ x }
    }
    ans
}

pub fn bar(n: i32, s: &[i32]) -> i32 {
    let mut ans = 0;
    for i in 0..n {
        ans += foo(true, i, s);
    }
    ans
}
```

```asm
foo:
        lea     eax, [rsi + rdi]
        ret
```

Loop in foo was optimized out

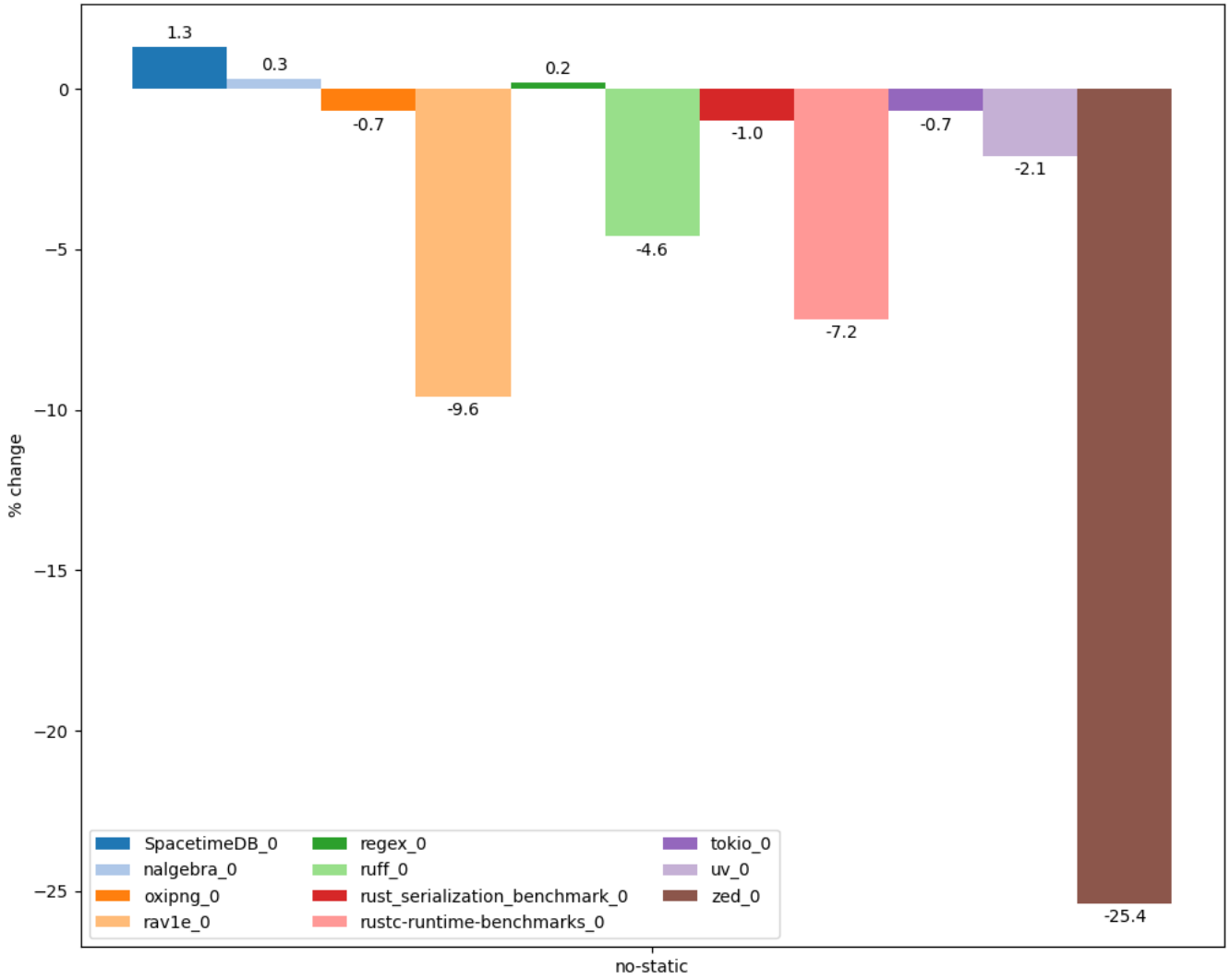
Comparison to C/C++

- Many authoritative coding guidelines suggest to localize functions:
  - Google C++ Style Guide
  - LLVM Coding Standard (Restrict Visibility)
  - C++ Core Guidelines
  - Chromium C++ Style Guide
- No way to localize private class methods in C++
  - Prototype optimization

Prevalence

- 80-90% of functions in Rust programs are local
  - Statistics collected via this patch
- In C/C++ results depend on the project:
  - 80-90% (clang, ffmpeg, git, opencv, bitcoin)
  - 40-50% (openssl, tmux, gcc, libuv)
  - (Statistics collected with Localizer tool)

Slowdown with disabled localization



Compiler patch

Conclusions

- Function localization allows compiler to generate much more performant code (over 5%)
- Rust has more optimal performance-wise defaults

Recommended readings

- Efficient C Tips #5 – Make ‘local’ functions ‘static’

No inplace-initialization

Detailed analysis

Overview

- C++ has placement-new mechanism for constructing objects in arbitrary memory buffer:

```
void *addr = some_alloc(sizeof(A));
A *ptr = new(addr) A(...);  // Or std::construct_at
```

- Complemented by perfect forwarding:

```
std::vector<LargeStruct> vec;
vec.emplace_back(large_struct_initializer);
```

- Rust lacks similar mechanism
  - Caused by mandatory object initialization
- Problem:

```
// Can compiler eliminate copy (from stack to heap) ?
let b = Box::new(LargeStruct { ... });
```

Why placement new is needed ?

```rust
const SIZE: usize = 1024 * 1024 * 1024;

fn main() {
    let b: Box<[usize]> = if cfg!(fast) {
        vec![15; SIZE].into_boxed_slice()
    } else {
        // Problematic copy
        Box::new([15; SIZE])
    };
    black_box(&b);
}
```

```
# Can compiler eliminate copy ?
$ LIMIT=$((8 * 1024))
$ rustc +baseline -O test.rs
$ (ulimit -s $ LIMIT; ./test)
thread 'main' has overflowed its stack
fatal runtime error: stack overflow

# Works with dedicated non-copying API
$ rustc +baseline --cfg fast -O test.rs
$ (ulimit -s $ LIMIT; ./test)
```

Why placement new is needed ?

```
struct A {
  std::array<int, 1024> vals = {};

  A() = default;
  A(const A &) = default;

  A(int val) {
    std::fill(vals.begin(), vals.end(), val);
  }
};

int main() {
  std::vector<A> aa;
  aa.reserve(NITER);

  for (size_t i = 0; i < NITER; ++i) {
#ifdef USE_PLACEMENT_NEW
    aa.emplace_back(i);
#else
    A a(i);
    aa.push_back(a);
#endif

    asm("" :: "r"(&aa) : "memory");
  }

  return 0;
}
```

USE_PLACEMENT_NEW
~10% faster !

Compiler optimizations

- Move/copy elision
  - Detects and removes redundant moves and copies
- Can work much more aggressively in Rust (compared to C++)
  - High-level IR (MIR)
  - Shallow move/copy (essentially move/copy in Rust is just a memcpy)
- Modern C++ compilers do not support complete copy elision optimization
  - Only limited optimizations like RVO, NRVO, etc.
  - Mainly due to lack of high-level IR (ClangIR to the rescue ?)
  - See Ultimate Copy Elision work by Roman Rusyaev (GitHub)
- Implementation in compiler:
  - MIR (high-level IR): CopyProp, DeadStoreElimination, DestinationPropagation, RenameReturnPlace
  - LLVM: MemCpyOptPass

Detailed analysis

Issues with move/copy elision

- Depends on compiler version
  - Not always monotonically
- Works only for release builds
  - Copies in debug builds are not optimized and may cause stack overflow or significant slowdown
- Language support for inplace-initialization is one of project goals in 2026:
  - In-place initialization

Workarounds

- Same as for mandatory initialization:
  - Use dedicated APIs to create uninitialized buffers and then initialize them:
  - E.g. Box/Rc::new_uninit, Vec::with_capacity/set_len
- Also other APIs e.g.
  - Vec::into_boxed_slice
  - Vec::resize_with, Vec::copy_with, HashSet::get_or_insert_with, etc.
    - (Increase chances for copy elision but do not guarantee it)

Conclusions

- Copy elision works much better in Rust than in C++
- But can’t fully compensate for lack of language support for inplace initialization
  - Like placement new and perfect forwarding
- Language developers are aware of this problem and try to solve it:
  - Rust Project Goals 2026: In-place initialization
  - Rust Project Goals 2026: MIR move elimination

Recommended readings

- TODO

Structure optimizations

Detailed analysis

Overview

- TODO

Example

- TODO

Overheads

- TODO

Compiler optimizations

- TODO

Workarounds

- TODO

Conclusions

- TODO

Recommended readings

- TODO

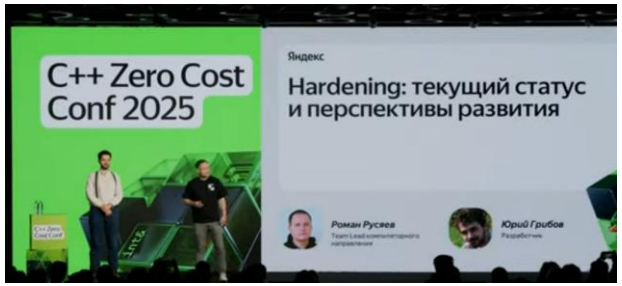
Arithmetic checks

Detailed analysis

Overview

- Arithmetic overflows may cause serious system crashes
  - ~1% CVE and 1.5% KEV in 2024
  - 23 place in Mitre CWE Top 25 2024 (8 in 2019 rating)
- Well-known incidents:
  - Therac-25 radiation therapy machine (1985)
  - Ariane 5 catastrophe (1996)
- History (pre-Rust):
  - `-ftrapv` added in 2000 in GCC (patch for -ftrapv option)
    - Wasn't well maintained and rotted quickly (e.g. BZ #35412 opened 2008 and still not fixed)
  - Works of John Regehr in 2010 (IOC)
  - UBsan created in 2014
    - State-of-the-art solution
- Additional info:
  - Hardening: current status and trends

J.R. Armstrong, Famous Failures Revisited



Example error (OpenSSH 3.3)

```
nresp = packet_get_int();
if (nresp > 0) {
  // Overflow to zero ...
  response = xmalloc(nresp*sizeof(char*));
  // ... and cause heap buffer overflow here
  for (i = 0; i < nresp; i++)
    response[i] = packet_get_string(NULL);
}
```

Default arithmetic checks in Rust

- Some (not very important...) arithmetic checks enabled by default
- Division by zero
- Signed overflow on division (INT_MIN / -1)
- Float-to-integer saturation
  - Can be disabled via -Zsaturating-float-casts=off
- Integer overflows in stdlib containers
  - Via assert!, checked_add, etc.

Important missing checks

- For historical reasons some important checks are missing
  - And can't even be enabled via flag
- E.g. overflow check on casing types via as
  - Need to use other APIs like try_into/ try_from

```rust
use std::convert::TryFrom;

fn main() {
    if cfg!(try_from) {
        i8::try_from(0x100i32).unwrap();
    } else {
        0x100i32 as i8;
    }
}
```

```
$ rustc +baseline test.rs -O
$ ./test
0
$ rustc +baseline --cfg try_from test.rs -O
$ ./test

thread 'main' panicked at tmp57.rs:5:32:
called `Result::unwrap()` on an `Err` value:
TryFromIntError(())
```

Integer overflow checks in Rust

- By default integer overflows in user code are checked only in debug builds
- Two's complement semantics in release
- Can be enabled in release via -C overflow-checks=on
- Reason: too large overheads
  - "Plan is to turn on checking by default *if* we can get the performance hit low enough and also that it should leave room in the future to move towards universal overflow checking if it becomes feasible" (Niko Matsakis, co-lead of Rust language design team)

Defined overflow

The Value of Undefined Behavior and Garbage In, Garbage Out

- Unlike C/C++ integer overflows are not Undefined Behavior
  - Panic or Two's complement
- This deprives optimizer of some important information
- Can reduce precision of some analyses (especially ScalarEvolution) and disable some optimizations in LLVM
  - See How undefined signed overflow enables optimizations in GCC

```
// For signed i1/i2 compiler
// can optimize
// inductive variables
// for &buf[i1] and &buf[i2].
// This is not possible
// for unsigned i1/i2.

T i1 = 0, i2 = 0;

for (;;) {
    int c1 = buf[i1];
    int c2 = buf[i2];
    if (c1 != c2)
        return c1 - c2;
    i1++;
    i2++;
}
```

Problem with inclusive ranges

- Loops with included upper bound are less optimized by compiler:

```
for i in 1..=n {
    total += i;
}
```

- Due to potential overflow (when `n==T::MAX`) compiler has to perform additional check:

```
while i <= n {
    total += i;
    if i >= n {
        break;
    }
    i += 1;
}
```

- Developers can help compiler by converting to exclusive loop:

```
for i in 1..(n+1)
```

- Examples: [Why isn't the for loop optimized better?](), [Comparing Pythagorean triples in C++, D, and Rust](), etc.

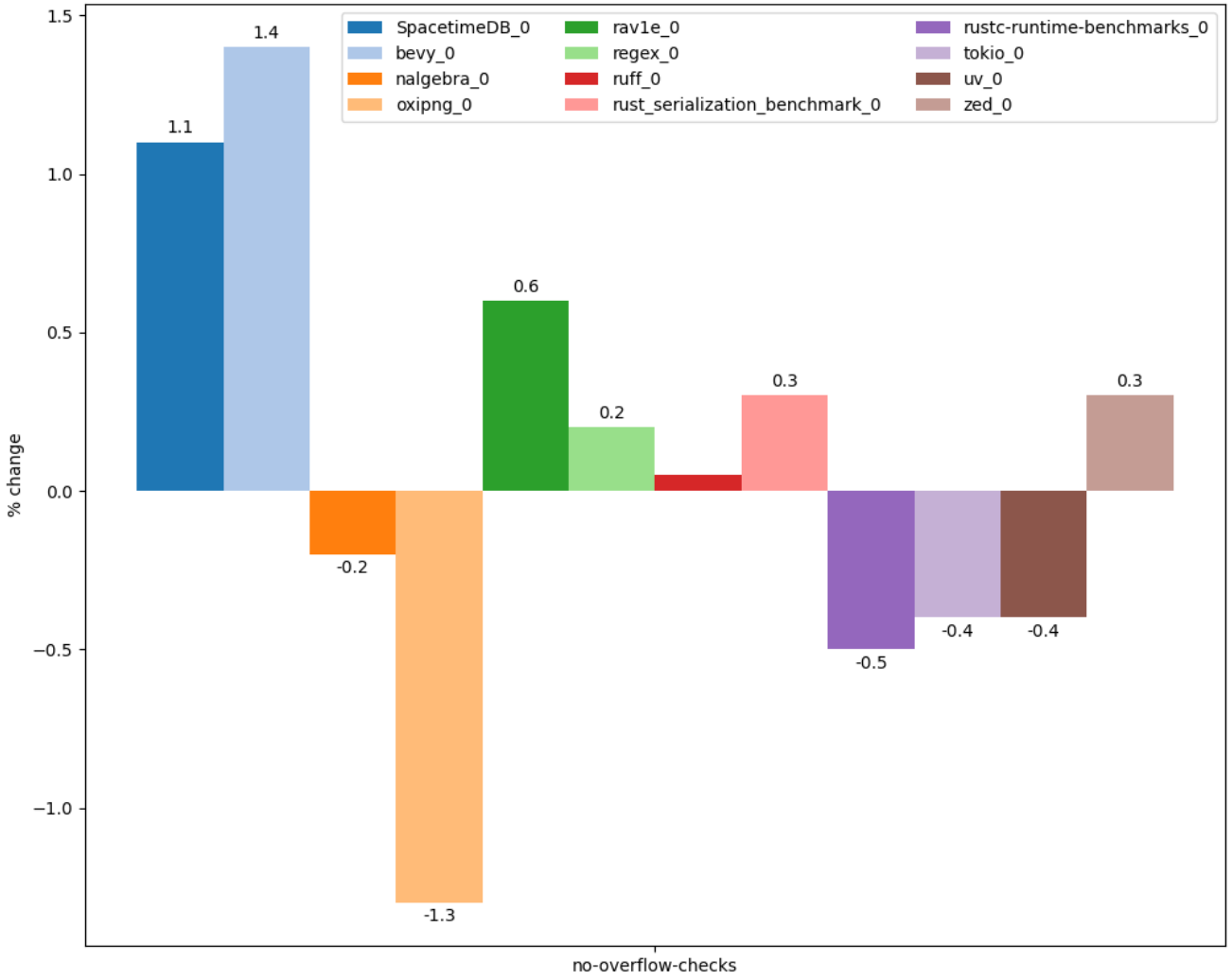
Other overheads

- Overheads for overflow checks:
  - Compare + predictable conditional branch (2 cycles + I$/BTB pressure)
  - Hurt other optimizations (vectorizer, etc.)
- The first (less important) issue could *potentially* be fixed with hardware support

```rust
#[no_mangle]
pub fn foo(xs: &[i32]) -> i32 {
    let mut ans = 0;
    for x in xs {
        ans += x;
    }
    ans
}
```

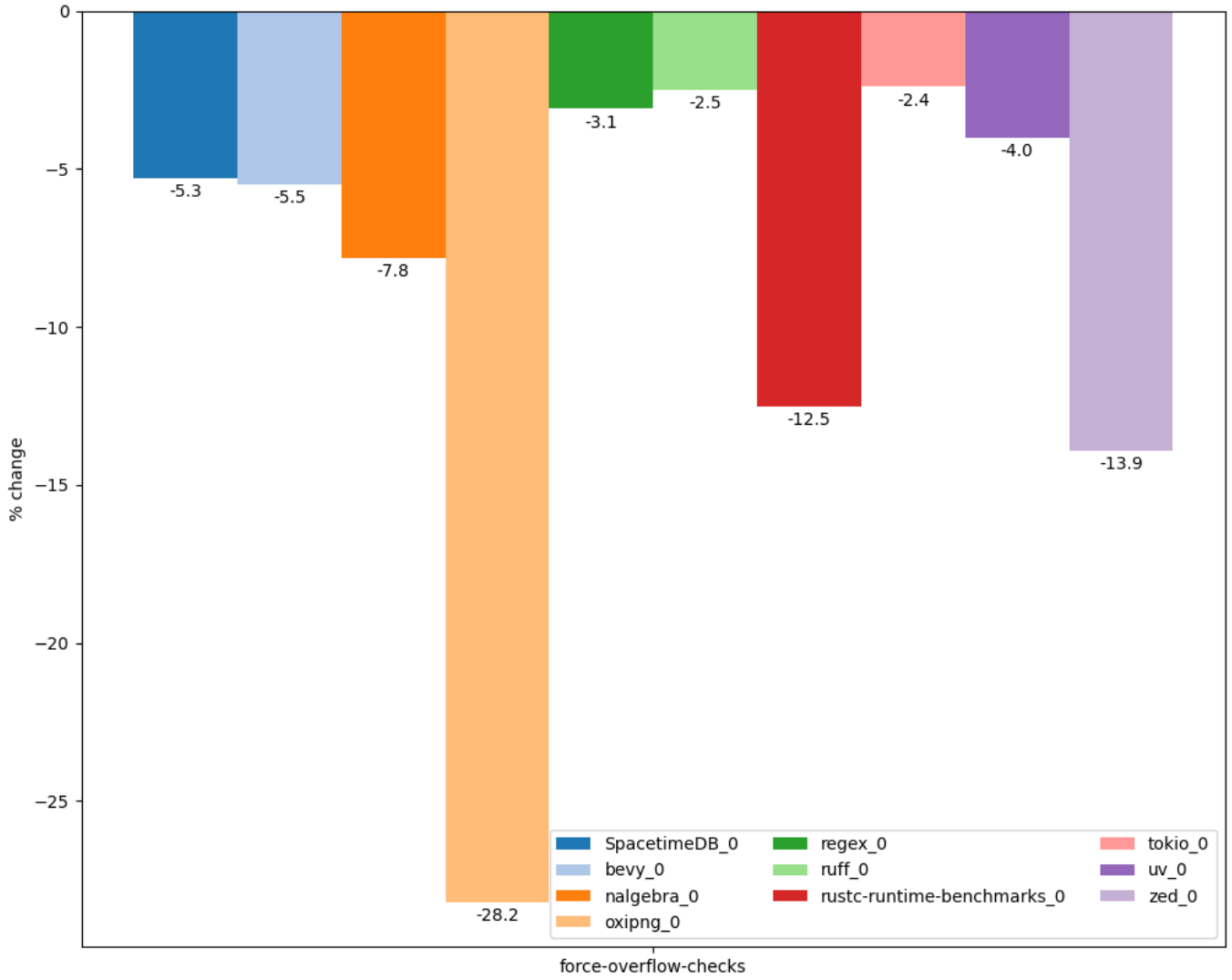
```
$ rustc ... test.rs
...
.LBB0_5:
        // Loop successfully vectorized
        movdqu  xmm2, xmmword ptr [rdi + 4*rax]
        paddd   xmm0, xmm2
        ...
$ rustc ... -Coverflow-checks=on test.rs
...
.LBB0_3:
        // Loop not vectorized
        add     eax, dword ptr [rdi + rcx]
        jo      .LBB0_6
        add     rcx, 4
        cmp     rsi, rcx
        jne     .LBB0_3
        ...
```

Improvements without default checks (division by zero, etc.)



Compiler branch

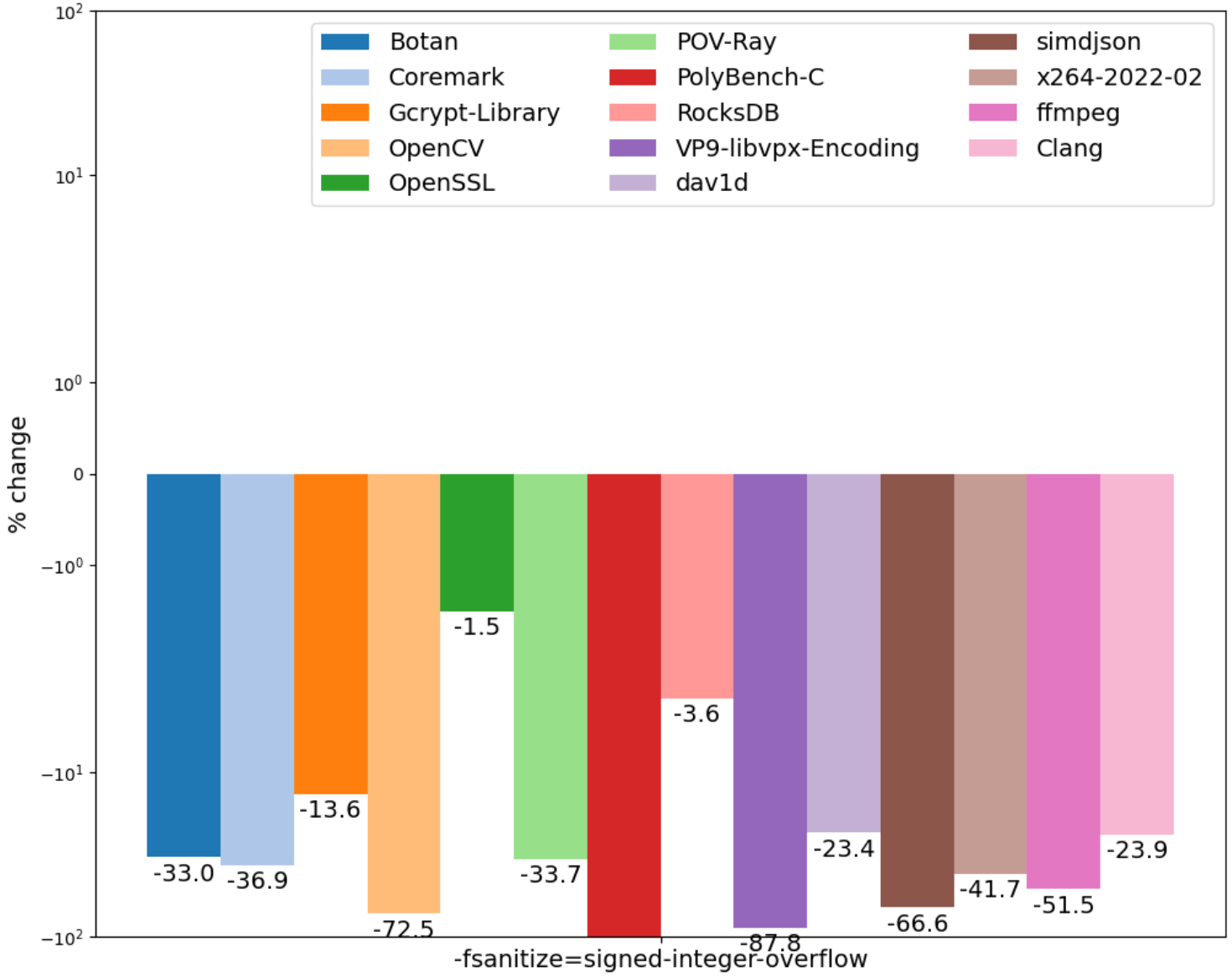
Slowdown with integer overflow checks



- +30% checks in rustc

Compiler branch

Slowdown in C++



- Similar of even larger overhead in C/C++ with UBsan

Compiler optimizations

- Integer overflows are not well optimized by compiler
  - E.g. Presburger arithmetic is decidable but complexity is O(2^2^N)
- In stage 2 rustc build compiler optimizes ~30% of arithmetic checks
  - Based on analysis of binary code

Workarounds

- Unsafe APIs:
  - unchecked_add, wrapping_add, etc.
  - to_int_unchecked
  - core::hint::unreachable_unchecked

```
if x <= i32::MIN >> 1 || x >= i32::MAX >> 1 {
    unsafe { std::hint::unreachable_unchecked() }
}
x * 2 / 2
```

  - (checks in stdlib *can not be disabled*)
- Special stdlib types:
  - NonZeroU32 to disable divide-by-zero checks
  - Wrapping<T> for overflows

Conclusions

- Integer overflow is not UB in Rust
  - Prevents compiler from performing some aggressive optimizations
- Rust makes non-obvious choice about default arithmetic checks
  - Some really important errors like lossy casts are not checked but zero divisions are
  - Rare but may cause slowdown of ~1-2%
- Integer overflow checks have very large overhead (up to 10%)
  - Same as UBsan
  - Unlikely to be enabled by default in future

Recommended readings

- RFC 560: Integer Overflow (also discussion and overview)
  - Attempt to enable integer overflow checks in release
- Understanding Integer Overflow in C/C++
  - Paper by John Regehr
- How undefined signed overflow enables optimizations in GCC
  - Paper about performance benefits of Integer Overflow UB in C/C++

Standard library

Detailed analysis

Overview

- Rust stdlib has more mandatory checks and in some cases uses more robust (and slow) algorithms compared to STL
  - Cargo build system greatly simplifies experiments with alternate implementations though
- On the other hand in other cases it can use faster container and algorithm implementations (compared to STL)

Standard library checks

- Stdlib contains a lot of mandatory checks (assert!, checked_add, etc.)
- Main types of checks:
  - Correctness of indices in containers (bounds checks, already considered above)
  - Size overflow checks in containers (capacity overflow)
  - Successful allocations in containers
  - Correctness of indices in UTF-8 container accesses (&str and String)
  - API-specific checks (comparator axioms, etc.)
- Similar checks are available in STL
  - Under _GLIBCXX_ASSERTIONS and _LIBCPP_HARDENING_MODE

Example

```rust
pub fn foo(length: usize) -> Vec<i32> {
    vec![-1; length]
}
```

```
        lea     r15, [4*rsi]
        xor     r13d, r13d
        mov     rax, rsi
        // Check that 4*length does not overflow
        shr     rax, 62
        jne     .LBB0_11
        // Check that 4*length <= isize::max
        movabs  rax, 9223372036854775804
        cmp     r15, rax
        ja      .LBB0_11
        ...
        // Check successful allocation
        call    qword ptr [rip +
_RNvCslKhMTRUAIjP_7___rustc12___rust_alloc@GOTPCREL]
        test    rax, rax
        je      .LBB0_11
```

Example

```rust
use std::io;

fn main() {
    let mut input = String::new();
    io::stdin()
        .read_line(&mut input)
        .unwrap();
    println!("{}", input);
}
```

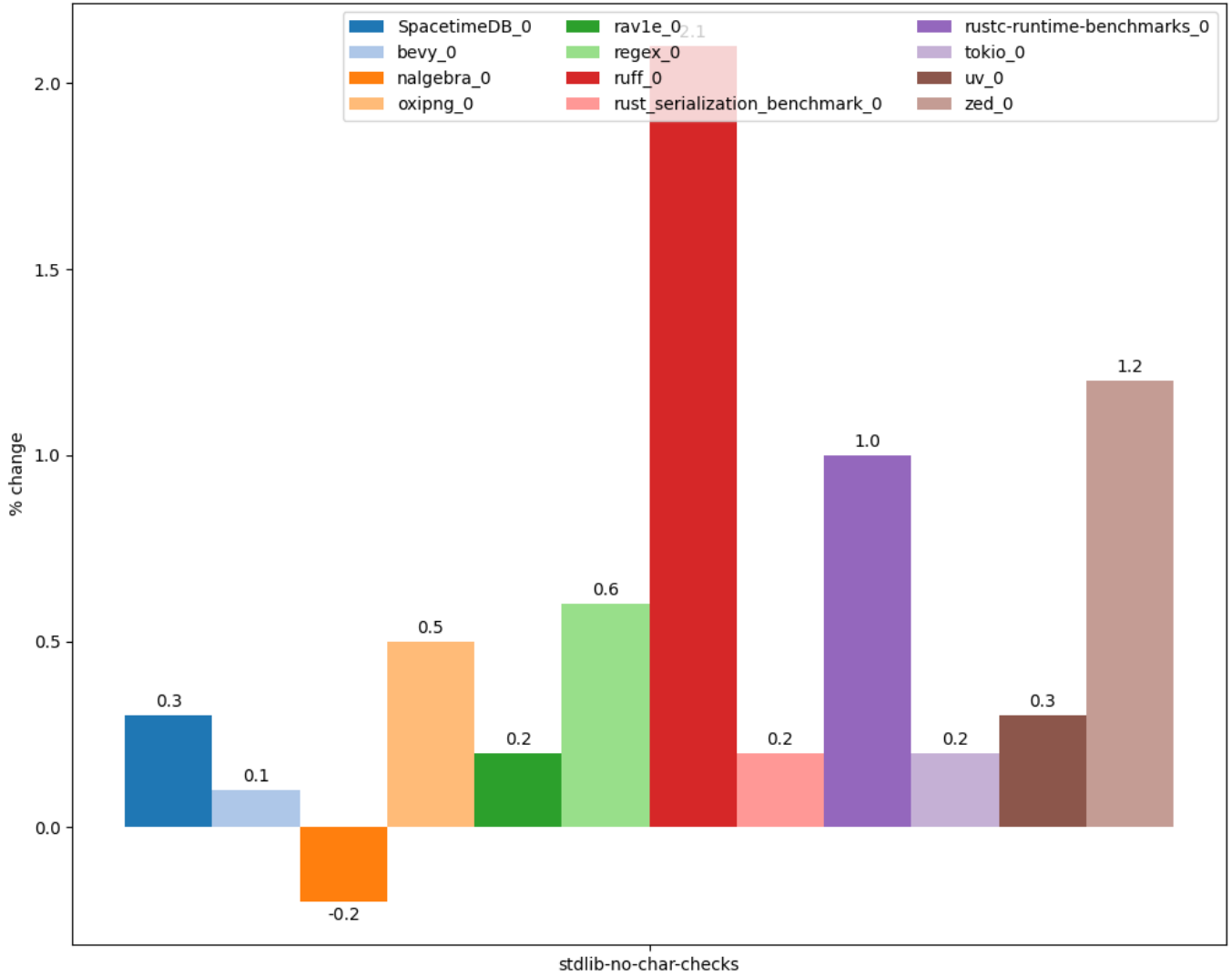
```
$ echo Привет | ./test
Привет

$ echo Привет | iconv -f UTF-8 -t KOI8-RU | ./test
thread 'main' panicked at test.rs:5:39:
called `Result::unwrap()` on an `Err` value:
Error { kind: InvalidData, message: "stream did not contain valid UTF-8" }
```

Other overheads

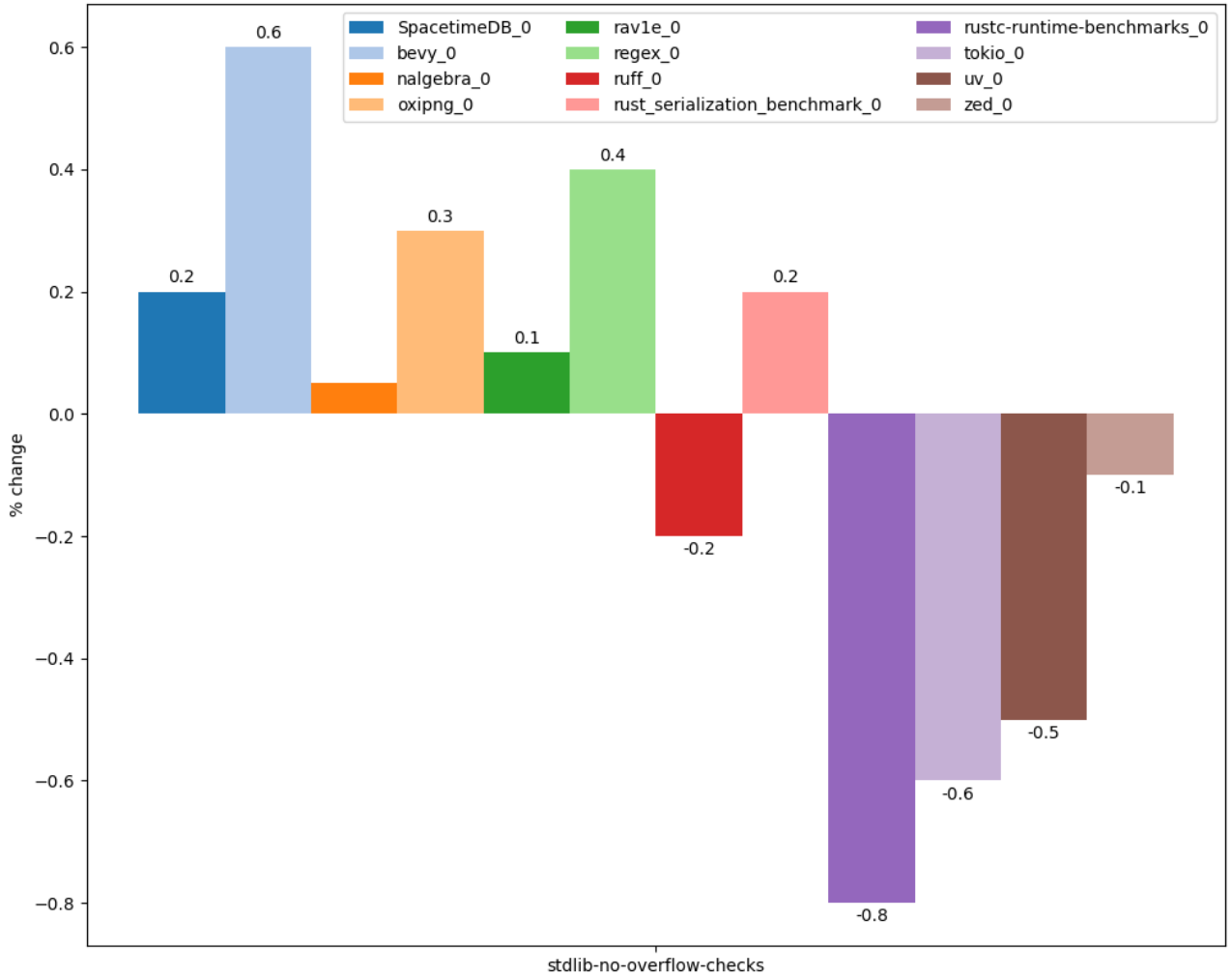
- Pseudo-random numbers generator is cryptographically secure and slower than rand(3) or std::mt19937
- Default hashing algorithm in hashtables (SipHash) is resilient against DoS attacks bit is slower than analogs in STL
  - 2x slowdown against fxhash in simple microbenchmark
- Rust IO is unbuffered by default
  - Without BufReader programs may be 10x slower
- io::stdout() is line-buffered for non-TTY output (rust #60673)
  - This may significantly slow down command-line tools (for cases like "./prog > file")
- print!/println! macros use locked access to stdout for stable output from parallel threads

Improvements with disabled UTF-8 checks



[Compiler branch]

Improvements with disabled container overflow checks



Compiler branch

Optimizations in standard library

- Hashtables:
  - No requirement for element reference validity after modification (as in std::unordered_map)
  - Allows to use open-addressing instead of chaining (fewer allocations, cache-friendly)
- Search trees use B-tree (instead of RB-tree as std::map)
  - Fewer allocations, cache-friendly
  - 4x speedup on simple microbenchmark
- Modern ILP-friendly sorting algorithms (ipnsort, driftsort)
  - Modern STLs are implemented to simplify vectorization when sorting primitive types
  - 65% speedup on simple microbenchmark

Conclusions

- TODO

Recommended readings

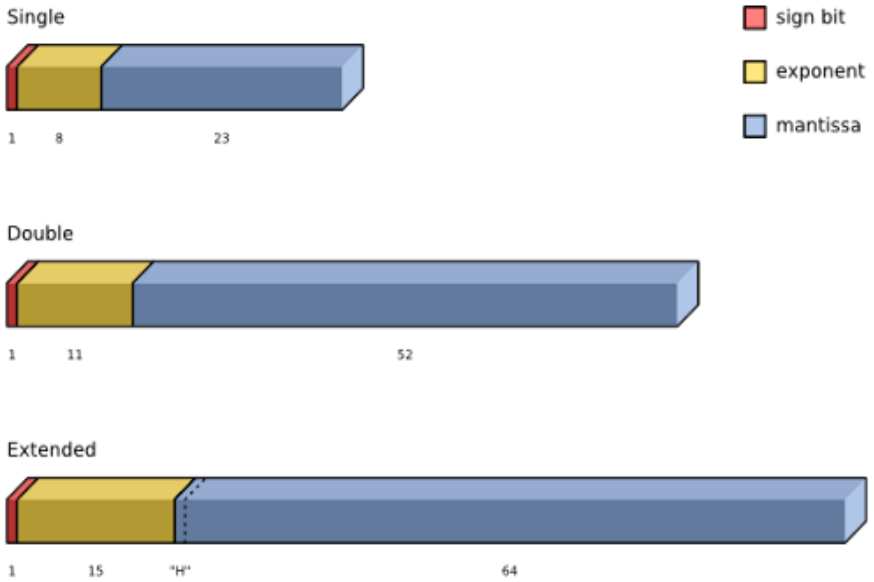
- Safety vs Performance. A case study of C, C++ and Rust sort implementations

Lack of fast-math

Detailed analysis

Floating point computations

- IEEE 754 standard
  - Types and their binary representation
  - Operations on types
  - Rounding
- Most architectures and languages follow the standard (mostly)
- Reasons for discrepancies:
  - Architecture specifics
  - More opportunities for optimizations
- Fast math – umbrella term for optimizations that are enabled under relaxed IEEE 754 rules



Types of fast-math flags

- Algebraic:
  - Associativity
  - Unsignedness of zero
  - Replacing division with multiplication
  - Fusion of operations (e.g. FMA)
- Disallowing non-standard values:
  - NaN, Infinity

```
double foo(double val) {
  return (val + 3.0) / 3.0;
}
```

```
.LCPI0_0:
        .quad   0x4008000000000000 # 3.0
foo:
        movsd   xmm1, qword ptr [rip +
.LCPI0_0]
        addsd   xmm0, xmm1
        divsd   xmm0, xmm1
        ret
```

clang-21 -O2

```
.LCPI0_0:
        .quad   0x4008000000000000 # 3.0
.LCPI0_1:
        .quad   0x3fd5555555555555 # 1/3
foo:
        addsd   xmm0, qword ptr [rip +
.LCPI0_0]
        mulsd   xmm0, qword ptr [rip +
.LCPI0_1]
        ret
```

clang-21 -O2 -freciprocal-math

Fast math in other languages

- Different languages have different approach towards IEEE 754
- C and C++ Standards disallow fast-math optimizations
  - Usually can be enabled with flags (e.g. -ffast-math)
  - Used e.g. in game engines
- Same applies for Swift language
- Java and C# allow to perform computations with higher precision than resulting type
- Go allows fusion of operations
  - Only if there are no explicit casts
- Fortran allows IEEE 754 violations to improve performance

Optimizations: Vectorization

- -fassociative-math and -fno-signed-zeros allow compiler to reorder loop iterations and vectorize loop

```c
#include <math.h>

float sum256(float *arr1, float *arr2) {
  float s;
  int i;
  s = 0.0f;
  for (i = 0; i < 256; i++) {
    s += arr1[i] * arr2[i];
  }
  return s;
}
```

```
.LBB0_1:
  movss   xmm1, dword ptr [rdi + 4*rax]
  mulss   xmm1, dword ptr [rsi + 4*rax]
  addss   xmm0, xmm1
  # Loop NOT vectorized
  inc     rax
  cmp     rax, 256
  jne     .LBB0_1
```

clang-21 -O2 -fno-unroll-loops

```
.LBB0_1:
  movups  xmm1, xmmword ptr [rdi + 4*rax]
  movups  xmm2, xmmword ptr [rsi + 4*rax]
  mulps   xmm2, xmm1
  addps   xmm0, xmm2
  # Loop vectorized
  add     rax, 4
  cmp     rax, 256
  jne     .LBB0_1
```

clang-21 -O2 -fno-unroll-loops
-fassociative-math -fno-signed-zeros

Optimizations: Algebraic simplifications

- Fast-math may allow more simplifications or compile-time evaluation
- -fassociative-math + -fno-signed-zeros

```
double foo(double val) {
  return 3.0 + val - 3.0;
}
```

```
foo:
  addsd   xmm0, qword ptr [rip + .LCPI0_0] # +3.0
  addsd   xmm0, qword ptr [rip + .LCPI0_1] # -3.0
  ret
```

clang-21 -O2

```
foo:
  ret
```

clang-21 -O2
-fassociative-math -fno-signed-zeros

- -fassociative-math + -fno-signed-zeros + -freciprocal-math

```
double foo(double val) {
  return 3.0 * val / 3.0;
}
```

```
foo:
  movsd   xmm1, qword ptr [rip + .LCPI0_0]
  mulsd   xmm0, xmm1 # * 3.0
  divsd   xmm0, xmm1 # / 3.0
  ret
```

clang-21 -O2

```
foo:
  ret
```

clang-21 -O2
-fassociative-math -fno-signed-zeros
-freciprocal-math

Optimizations: Fused instructions

- Some architectures support fusion of floating point instructions
  - E.g. Fused Multiply Add in x86-64
- Rounding of these instructions may be different from sequence of roundings of their primitive operations
- Enabled via -ffpcontract=on flag

```
double foo(double *val) {
  double acc = 1;
  for (int i = 0; i < 100; ++i) {
    acc = acc * val[i] + val[i + 1];
  }
  return acc;
}
```

```
.LBB0_1:
        mulsd   xmm0, xmm1 # multiply
        movsd   xmm1, qword ptr [rdi + 8*rax]
        addsd   xmm0, xmm1 # add
        inc     rax
        cmp     rax, 101
        jne     .LBB0_1
        ret
```

clang-21 -O2 -fno-unroll-loops

```
.LBB0_1:
        vmovsd  xmm2, qword ptr [rdi + 8*rax]
        vfmadd213sd     xmm0, xmm1, xmm2 # fused multiply-add
        inc     rax
        vmovapd xmm1, xmm2
        cmp     rax, 101
        jne     .LBB0_1
        ret
```

clang-21 -O2 -fno-unroll-loops -ffp-contract=on -march=haswell

Optimizations: Finite only

- -ffinite-math-only enables optimizations that rely on lack of NaN or Inf values (and corresponding runtime exceptions)
- This allows to
  - Remove NaN and Inf checks
  - x == x => true
  - x / x => 1
- With -fno-signed-zeros:
  - x – x => 0
  - 0 / x => 0
  - x * 0 => 0

```
bool foo(double val) {
  return val == val;
}
```

```
foo:
  ucomisd xmm0, xmm0
  setnp   al
  ret
```

clang-21 -O2

```
foo:
  mov     al, 1
  ret
```

clang-21 -O2
-ffinite-math-only

```
double foo(double val) {
  return val * 0;
}
```

```
foo:
  xorpd   xmm1, xmm1
  mulsd   xmm0, xmm1
  ret
```

clang-21 -O2

```
foo(double):
  xorps   xmm0, xmm0
  ret
```

clang-21 -O2
-ffinite-math-only
-fno-signed-zeros

Issues with fast-math: Checks

- Usage of Inf or NaN with -ffinite-math-only is UB
- Explicit checks for such values will be removed by compiler

```c
#include <math.h>

int foo(float f) {
  return isnan(f);
}
```

```
foo:
  xor     eax, eax
  ucomiss xmm0, xmm0
  setp    al
  ret
```

clang-21 -O2

```
foo:
  xor     eax, eax
  ret
```

clang-21 -O2
-ffinite-math-only

Issues with fast-math: Rounding

- Some algorithms use special order of computations to reduce rounding error
- With -fassociative-math compiler may ignore this order and “optimize” these algorithms as if there is no rounding
- E.g. Kahan algorithm

```c
float sum(float *arr) {
  float s;
  int i;
  s = 0.0f;
  for (i = 0; i < 256; i++) {
    s = s + arr[i];
  }
  return s;
}
```

```
sum_kahan:
  lea     rax, [rdi+1024]
  pxor    xmm0, xmm0
.L2:
  addss   xmm0, DWORD PTR [rdi]
  add     rdi, 4
  cmp     rdi, rax
  jne     .L2
  ret
```

clang-21 -O2 -ffast-math

```c
float sum_kahan(float *arr) {
  float s, c, t, y;
  int i;
  s = c = 0.0f;
  for (i = 0; i < 256; i++) {
    y = arr[i] - c;
    t = s + y;
    c = (t - s) - y;
    s = t;
  }
  return s;
}
```

Rust approach

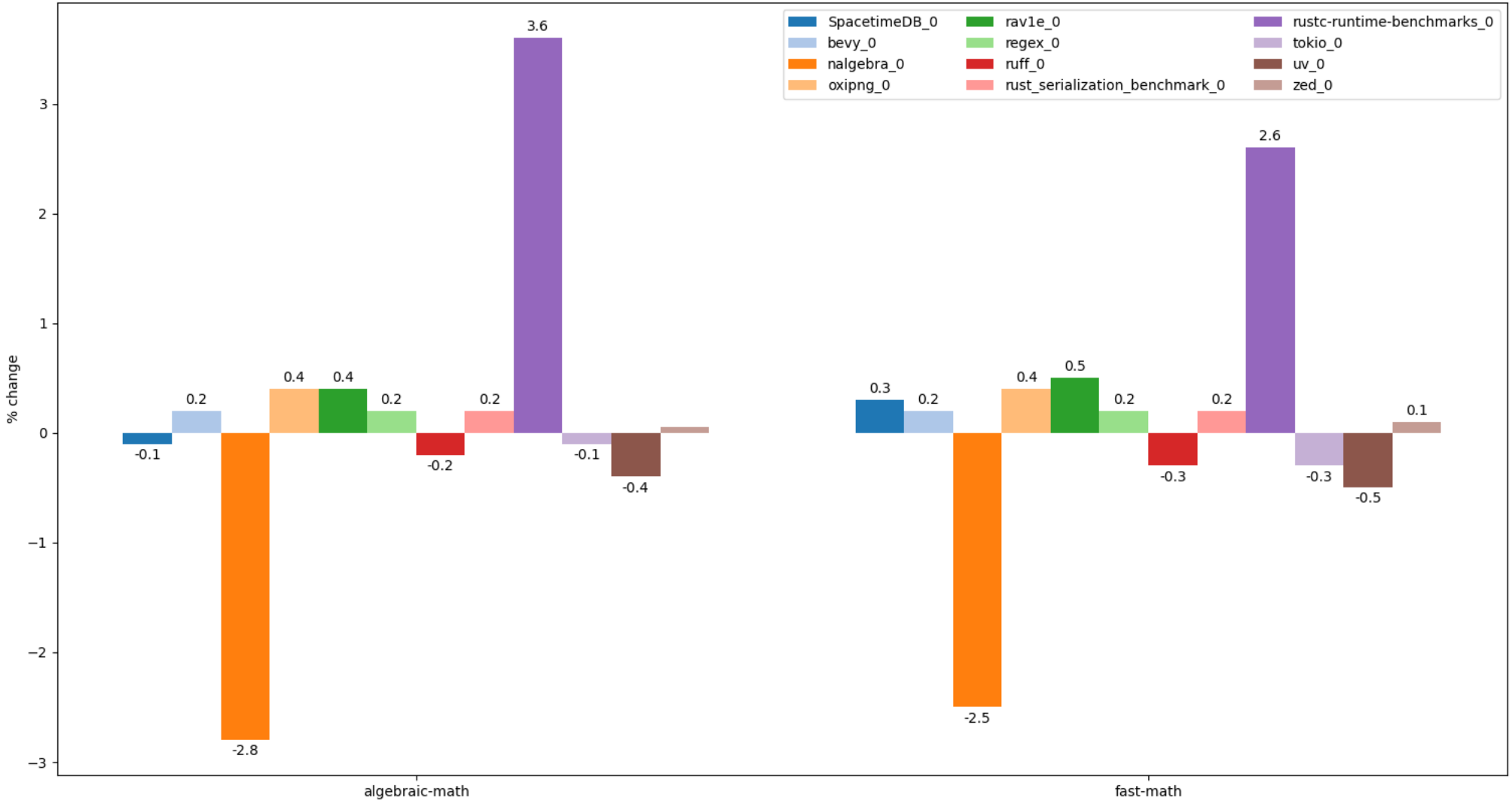
- Developers of Rust rejected to enable opportunities for global fast-math optimizations
- Due to potentially unsafe usage of these optimizations by users (especially in project dependencies)

> “Everything about this flag fundamentally clashes with the idea of robust, compositional system design. It was a terrible mistake to ever add it to C compilers, and Rust should not repeat that mistake. Rust should instead explore alternative, less fragile ways of exposing those semantics.” (Ralf Yung, rust #21690)

Workarounds: Intrinsics

- Rust provides dedicated intrinsics for enabling fast-math optimizations locally
  - std::intrinsics::f{op}_fast
  - std::intrinsics::f{op}_algebraic
  - f(16/32/64)::algebraic_{op}
- Currently require nightly (unstable) compiler build

Performance with fast-math



[Compiler branch](#) [Compiler branch](#)

Conclusions

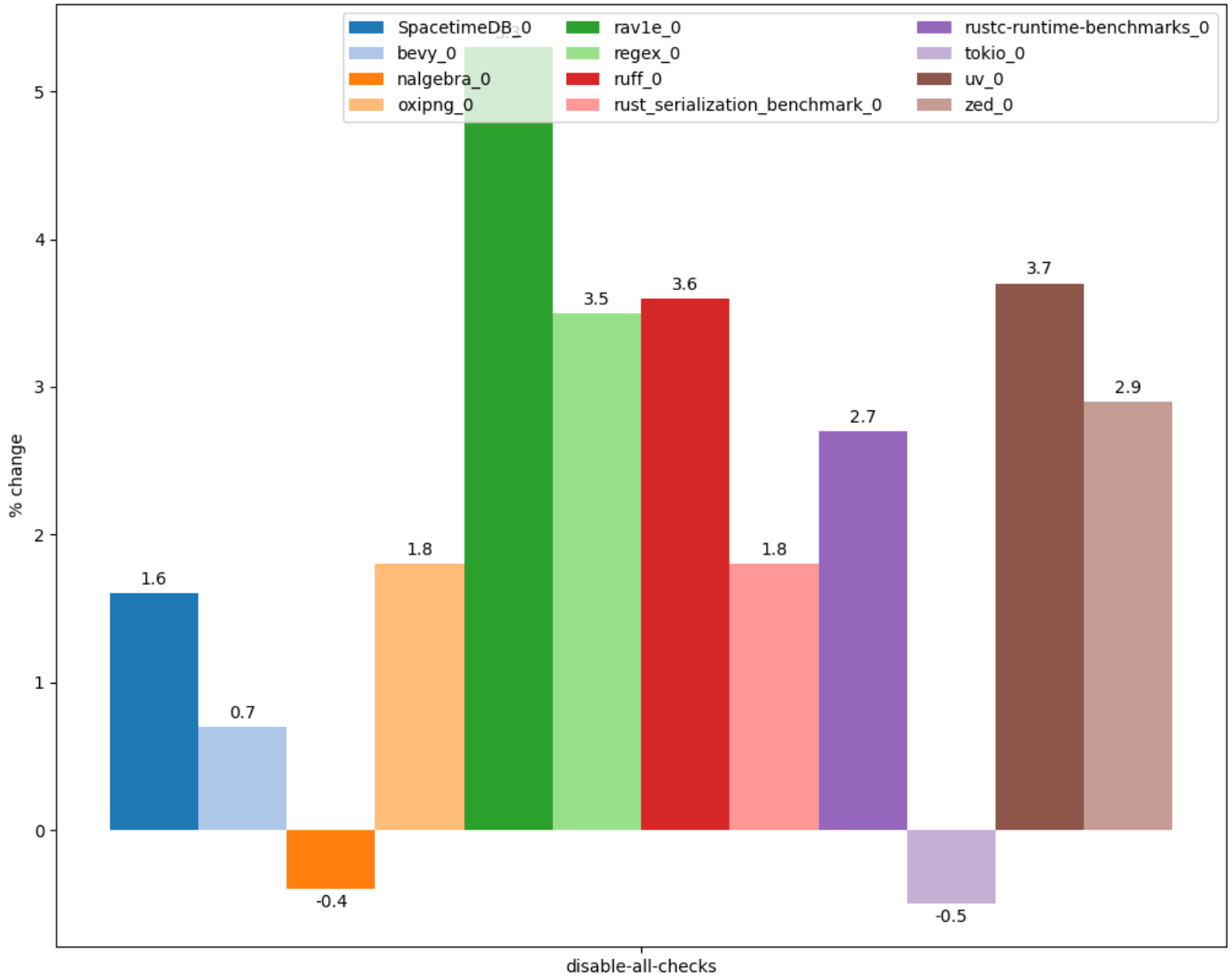
- TODO

Recommended readings

- TODO

Conclusions

Speedup with disabling (most) Rust checks



- Disabled bounds checks, overflow checks in stdlib containers, UTF-8 string checks
- Overheads for redundant initializations remain (expecting >= 1%)

Compiler branch

Conclusions

- Rust community keeps balance between safety and performance:
  - Main focus is on memory safety guarantees
  - Other checks are enabled if they do not hurt performance
  - For example integer overflow errors are disabled by default
- Overhead of runtimes checks depends on project and is around 3-6%
  - TODO: compare with Hardened C/C++
- Language features which affect performance the most:
  - Bounds checks, mandatory initialization, lack of inplace initialization
- These checks will most likely be further optimized in future versions of Rust and LLVM
- But their negative effects will never be completely avoided (at least in current language version)
- So they will always require workarounds and non-idiomatic code to reach optimal performance

What we didn’t talk about …

- Codegen units
- Implementation of panics (exceptions)
  - Overall similar to C++ exceptions
- GC-sections by default
- ABI features
- Disabled PLT
- Implementation of virtual methods
- Stack Clashing checks

Questions ?

Latest version of this presentation: https://github.com/yugr/rust-slides/blob/main/EN.pdf

Preparation of this talk

- We collected and analyzed over 350 papers, blogposts and discussions about the topic (see materials)
- Overall time spent on preparation (Mar 2026):
  - > 2.5 man-months
- Analysis was conducted for
  - Rust 1.87.0 (May 2025)
  - Clang llvmorg-20.1.7 (June 2025)

Rust benchmarks

- We used benchmarks of large open-source Rust projects from various domains:
  - Gamedev (Bevy, Veloren)
  - Databases (SpacetimeDB)
  - Text processing (Zed, Ruff, regex)
  - Codecs (rav1e, oxipng)
  - Linear algebra (nalgebra)
  - Asynchronous runtime (tokio)
  - Packet managers (uv)

Rust benchmarks

- Servers have been set up according to uInit methodology
- Used standard Criterion benchmarks in each project (cargo bench), averaged with geomean
  - Some especially slow tests in oxipng were disabled
  - To counteract system noise we took minimum of average from several runs
  - Note that 1% geomean may mean that
    - All tests in project slowed down by 1%
    - 10% of tests slowed down by 10%
  - Some tests could slow down (or speedup) by more than 10%
- Further details here

C++ benchmarks

- Servers have been set up according to uInit methodology
- Used Clang llvmorg-20.1.7
- Llvm-bench (benchmarks/cpp/llvm-bench)
  - Measured median time of -O2 compilation of CGBuiltin.cpp (the largest and slowest-to-compile file in LLVM) with Clang
  - Tested Clang was built with itself
- Ffmpeg-bench (benchmarks/cpp/ffmpeg-bench)
  - Based on Stack Smashing Protection and Its Performance Impact
  - Measured median time to convert FLV to MP4
- Phoronix Test Suite (benchmarks/cpp/PTS)
  - Popular benchmark suite from phoronix-test-suite.com
  - Measured botan, bullet, coremark, c-ray, dav1d, gcrypt, gmpbench, luajit, nginx, opencv, openssl, polybench-c, povray, rocksdb, simdjson, vpxenc, x265 (see #55)
  - Used median time of benchmark
  - Ignored tests with Stdev > 0.5% or changes < 1%
- To counteract system noise we took minimum of average from several runs

Hardware

- All benchmarks (except no-static) were run on Intel Xeon E-2236
  - With fixed 2 GHz frequency
  - With Ubuntu 24.04 OS

TODO

- Why InterPAA is more complex than IntraPAA ? Can’t we just inline everything ?
- Move info about benches to start
- Add info about focus of each bench (numerics, graphs, IO, etc.)
- Remove in reduced version:
  - Reading mats
  - Inplace init
  - Workarounds
  - Fast math examples